DOT-Design

Version 0.1.0, 2026-10-12

Overview

DOT-Design defines the portable model and reference tooling used to describe, validate, deploy, and compose Dynamic Open Trust (DOT) components in the MARE 6G security ecosystem.

A DOT packages four concerns without collapsing them into one runtime-specific description:

- **capability** — Skills describe what the component does;
- **contract** — OpenAPI, AsyncAPI, Protobuf, or GraphQL documents describe authoritative interactions;
- **composition** — Anchors give selected Contract operations stable identities and `PROVIDES` or `REQUIRES` roles; and
- **deployment** — Runtimes and deployment manifests describe where and how the component can run.



`dot.json` is the deployable DOT definition and the only source inspected by `dotx deploy`. Connections between deployed DOT instances are described in a separate composition document. Concrete addresses, credentials, and live policy results never become Anchor fields.

Documentation map

Read	When you need to
Getting started	Build the project, validate a DOT, inspect a composition plan, or generate this documentation.
Architecture	Understand repository boundaries, sources of truth, runtime/deployment separation, and the composition resolution path.
DOT Anchors and Composition	Author Anchors, bind them to services, connect DOT instances, materialize configuration, handle security inputs, and operate a composition.
Verification and examples	Choose conformance, dry-run, source-adapted, or live evidence and find the executable MARE PoC scenarios.
DOT-Domain	Work with the Java aggregate, value objects, validation, Contracts, Runtimes, and deployment manifests.
DOT-Registry	Use the read API, resolved-Anchor representation, error contract, persistence, or Registry tests.

Repository components

Component	Responsibility	Build ecosystem
`DOT-Domain`	Java domain model and validation library. It is not a web application.	Maven / Java 26
`DOT-Registry`	Spring Boot service for MongoDB persistence, Anchor discovery, Contract retrieval, and normalized Contract-element resolution.	Maven / Spring Boot 4
`DOT-CLI`	Independent `dotx` CLI for authoring support, validation, inspection, deployment, runtime lifecycle, and the provisional composition executor.	Go 1.25+
`DOT-Schema`	Generated JSON Schema for `dot.json`, tracked directly in this repository.	JSON Schema
`DOT-Repository`	Git submodule containing reusable DOT definitions and samples.	Git submodule
`DOT-Skills`	Git submodule containing the Skills catalog.	Git submodule
`testing`	Repository-level conformance, composition, interoperability, and end-to-end evidence. It is test infrastructure, not a Maven module.	Maven, Go, shell, container tools

Supported Contract standards

Standard	Anchor selector
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object
Protobuf	Fully qualified service method
GraphQL	Root-field schema coordinate

Contractless Anchors remain available for interactions that cannot be described by one of these standards. They must declare direction, protocol, and media type explicitly.

Maturity and scope

The DOT-owned Anchor model, service-port binding, generated schema, Registry resolution, and conformance fixtures are release-candidate surfaces. The separate `dot.mare6g.eu/v1alpha1` composition document and `dotx compose` executor remain a reference implementation.

The reference executor is intentionally fail-closed. It does not claim to be a general multi-host orchestrator, trust-policy engine, active security scanner, or replacement for Kubernetes, Helm, Docker, Vagrant, or an SFC implementation. See the supported composition profile for the exact boundary.

First steps

```
# Domain unit tests; no MongoDB required
mvn -pl DOT-Domain test

# Install the pinned CLI checkout on PATH
task cli:install

# Validate and inspect the canonical composition example
dotx compose validate DOT-Testing/e2e/api-gateway-oidc/scenario.yaml
dotx compose plan DOT-Testing/e2e/api-gateway-oidc/scenario.yaml

# Run the complete Anchor/composition repository gate
task testing:conformance
```

Continue with Getting started for prerequisites and command context.

Project links

- MARE project
- MARE proof-of-concept catalogue
- DOT-Design source repository

Getting started

This page establishes a small, reproducible development path. Start with the domain and CLI checks, which do not require MongoDB, before running the Registry integration tests or live deployment scenarios.

Prerequisites

Tool	Required for	Project baseline
Java and Maven	`DOT-Domain`, `DOT-Registry`, schema generation, and Java tests	Java 26; Maven 3.9+
Go	Building and testing `dotx`	Go 1.25+
MongoDB	Running `DOT-Registry` and its integration tests	Accessible on `localhost:27017` by default
Task	Repository command aliases	Optional but recommended
Docker, Kubernetes tools, Helm, or Vagrant	Only the matching deployment adapters and live scenarios	Install only what you intend to execute
Antora, the Antora PDF extension, Asciidoctor PDF, and Mermaid CLI	Building the HTML/PDF documentation site and Mermaid diagrams	See Build the documentation

Check out the repository

```
git clone https://github.com/MARE-EU-Project/DOT-Design.git
cd DOT-Design
git submodule update --init --recursive
```

The source repository and its submodules may require access granted by the MARE-EU-Project organization. `DOT-Schema` is tracked directly and does not require a separate checkout.

Verify the domain model

The domain module uses plain JUnit tests and does not start Spring or require MongoDB:

```
mvn -pl DOT-Domain test
```

Use the full Maven reactor only when MongoDB is available for Registry tests:

```
mvn clean install
# equivalent Task alias
```

```
task mci
```

The build generates CycloneDX JSON and XML software bills of materials during the Maven `verify` phase.

Install and test dotx

`DOT-CLI` is an independent Go module and is not part of the Maven reactor:

```
task cli:build
task cli:test
task cli:install
dotx --version
```

`task cli:install` installs the pinned checkout into `GOBIN` or `GOPATH/bin` and installs Bash completion. Keep that directory on `PATH`; project tasks, test harnesses, examples, and CI all invoke the same `dotx` command rather than a repository-local binary.

Validate a DOT

`dotx validate` reads `dot.json` at the project root, falling back to `dot/dot.json`. The selected file is the unique DOT deployment definition; unreferenced Dockerfiles, Compose files, Kubernetes manifests, Helm charts, and scripts are not inferred.

```
dotx validate DOT-Testing/conformance/anchors/valid \
  --schema DOT-Schema/schema.json
dotx inspect DOT-Testing/conformance/anchors/valid --json
```

Validation checks the JSON Schema and domain relationships. Inspection provides a normalized, machine-readable view without deploying anything.

Validate and plan a composition

The canonical example connects Keycloak, APISIX, and an OAuth client:

```
dotx compose validate \
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml

dotx compose plan \
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml

dotx compose plan \
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml --json

dotx compose apply \
```

```
  DOT-Testing/e2e/api-gateway-oidc/scenario.yaml --dry-run
```

All four commands above are non-mutating. `validate` answers whether references and compatibility can be resolved; `plan` shows the exact instances, endpoints, security status, and materializers; `apply --dry-run` renders supported runtime actions without writing composition state.

Read Validation, planning, and lifecycle before a live apply.

Run DOT-Registry

Start MongoDB first, then run the Spring Boot application:

```
mvn -pl DOT-Registry spring-boot:run
```

The default application database is `dots`. Integration tests use the separate `dots-test` database:

```
cd DOT-Registry
./mvnw test
```

The Registry currently exposes read-only Anchor and Contract endpoints. See DOT-Registry for the routes and response model.

Run the composition conformance gate

```
task testing:conformance
```

This gate combines Java domain and Registry tests, Go tests, generated-schema validation, fixture inspection, composition validation/planning, and dry-run scenario checks. It does not claim that external partner systems or production networks were exercised.

Build the documentation

Install the documentation toolchain once:

```
task docs:setup
```

Then generate the Antora HTML site and PDF export:

```
task docs:check
task docs:build
```

Generated artifacts are written under `DOT-Docs/build/site` and are intentionally ignored by Git. To build and serve the site locally:

```
task docs:run
```

Open `http://localhost:8000`. Do not edit files under `DOT-Docs/build`; update the AsciiDoc sources instead.

Mermaid sources are stored as Antora attachments and rendered locally by `task docs:diagrams`. The renderer produces SVG for the HTML site and high-resolution PNG for PDF export; both outputs are committed so a clean checkout can resolve every Antora resource. Each source includes an accessible title and description, and each AsciiDoc image macro supplies alternative text.

`task docs:build` regenerates the diagrams before building either output. The toolchain is version-pinned and does not send architecture content to an external rendering service. Edit the `.mmd` source, never the generated SVG or PNG, then run the full documentation build to verify both renderers.

Where to continue

- Architecture for system and ownership boundaries.
- DOT Anchors and Composition for the model overview and task-oriented composition guides.
- Verification and examples for evidence levels and MARE PoC scenarios.
- DOT-Domain for Java model details.
- DOT-Registry for persistence and read APIs.

Architecture

DOT-Design separates reusable component description from deployment-instance topology. That separation allows a DOT to be authored and validated once, then instantiated in multiple compositions without embedding addresses, credentials, or orchestrator-specific connection objects in the DOT aggregate.

The diagrams use a consistent visual language: blue identifies authored or persistent artifacts, teal identifies validation and resolution activities, amber identifies decisions or incomplete evidence, green identifies successful outcomes, and red identifies rejection. Labels and line styles carry the same meaning when the documentation is printed without color; dashed lines indicate optional, indirect, or evidence-only relationships.

System context

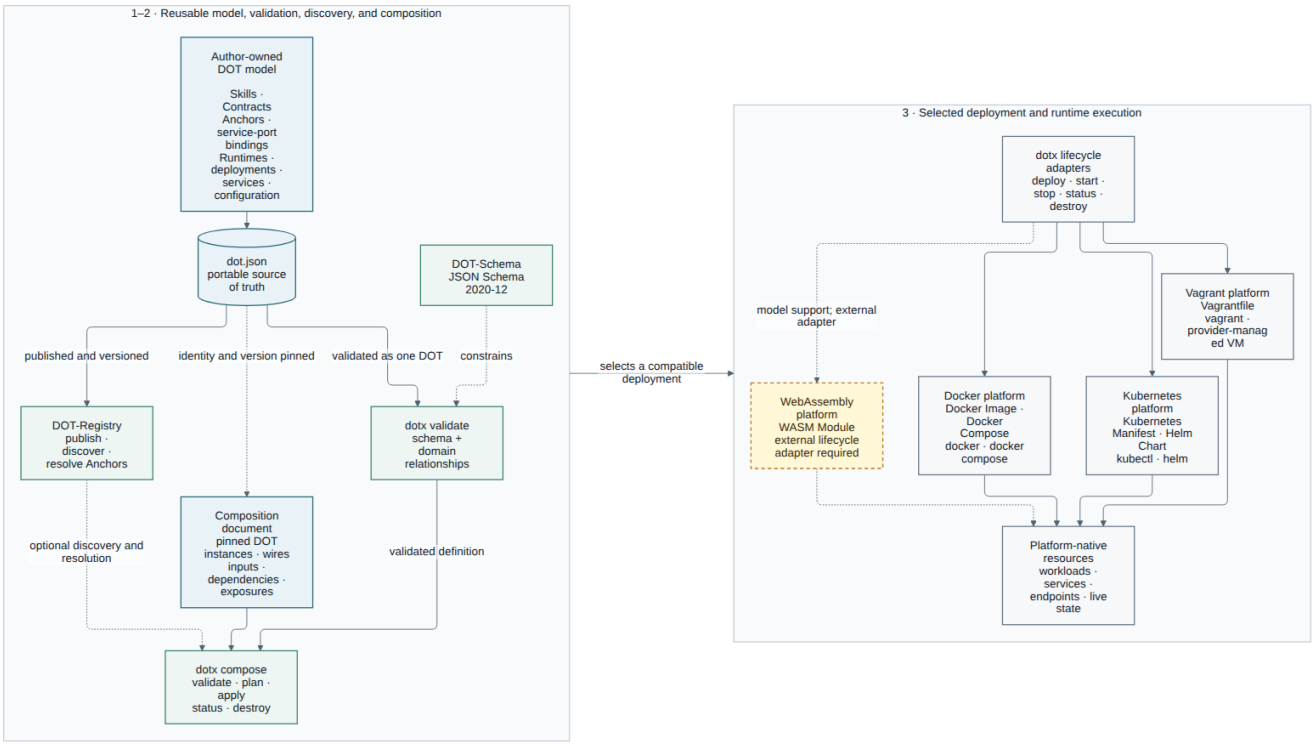


Figure 1. DOT-Design system context and runtime support

The Registry is a discovery and normalization service. `dotx` is the reference authoring, deployment, and composition client. Platform-native systems remain responsible for workload execution and live state.

The runtime portion deliberately distinguishes **where** a workload runs from **how** it is deployed. The domain model supports four `PlatformEngine` values and six compatible manifest mappings:

- Docker accepts `DOCKER_IMAGE` and `DOCKER_COMPOSE`;
- Kubernetes accepts `KUBERNETES_MANIFEST` and `HELM_CHART`;
- Vagrant accepts `VAGRANTFILE`; and
- WebAssembly accepts `WASM_MODULE`.

The current `dotx` lifecycle implements Docker Image, Docker Compose, Kubernetes manifest, Helm chart, and Vagrantfile execution. `WASM_MODULE` is a portable model surface, but deploying it requires a future or external WebAssembly lifecycle adapter. This distinction prevents schema support from being mistaken for reference-executor support.

Source-of-truth boundaries

Concern	Authoritative source	Explicitly excluded
DOT identity and supported capability	`dot.json`: `id`, `name`, `version`, Skills, and Contracts	Composition-local instance names and live state
Contract semantics	OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract payload	Repeated protocol, media, schema, or security fields on a contract-backed Anchor
Logical composition point	`Dot.anchors[]`	Hosts, URLs, credentials, placement, health, and policy decisions
Deployable endpoint declaration	`DeploymentService.ports[]` and `anchorBindings[]`	Allocated addresses, forwarded ports, and certificates
Runtime configuration surface	`DeploymentService.config`	Operator-supplied values already applied to one deployment
Multi-DOT topology and readiness	Composition document wires and `dependsOn`	Changes to either participating `dot.json`
Runtime state	Platform-native resources plus private `dotx` composition state	Contract or Anchor metadata

Repository architecture

```
DOT-Design
 ├──── DOT-Domain       Java model and validation
 ├──── DOT-Registry     Spring Boot read API and MongoDB persistence
 ├──── DOT-Schema       generated dot.json schema
 ├──── DOT-CLI          independent Go CLI and reference executor
 ├──── DOT-Repository   reusable DOT definitions (submodule)
 ├──── DOT-Skills       Skills catalogue (submodule)
 ├──── testing          conformance, composition, interoperability, and E2E evidence
 └──── docs             Antora sources and supplemental UI
```

The Maven reactor contains `DOT-Domain` and `DOT-Registry`. `DOT-CLI` has its own Go module and lifecycle. `testing` is deliberately outside both build-module hierarchies.

DOT aggregate

`Dot` is the aggregate root. Its primary relationships are:

```
Dot
 ├──── Skills[]                human-readable capabilities
 ├──── Contracts[]             authoritative interaction definitions
 ├──── Anchors[]               stable logical composition points
 └──── Runtimes[]
     ├──── Platform            where execution occurs
     └──── Deployments[]
         ├──── DeploymentManifest   how deployment occurs
         └──── DeploymentService[]
             ├──── Contract references
             ├──── ServicePort[]
             ├──── AnchorBinding[]
             └──── DeploymentConfig
```

`Skill` and `Contract` are standalone MongoDB documents referenced by ID. Anchors, Runtimes, Deployments, services, bindings, and configuration declarations are embedded values owned by the DOT aggregate.

The class-level `@ValidAnchor` constraint delegates aggregate traversal to `AnchorValidator`. It enforces Anchor identity, Contract membership, selector syntax, and service-port binding consistency without placing cross-reference logic in `Dot`.

Contract, Anchor, binding, and wire

These four concepts form a resolution chain but have different owners:

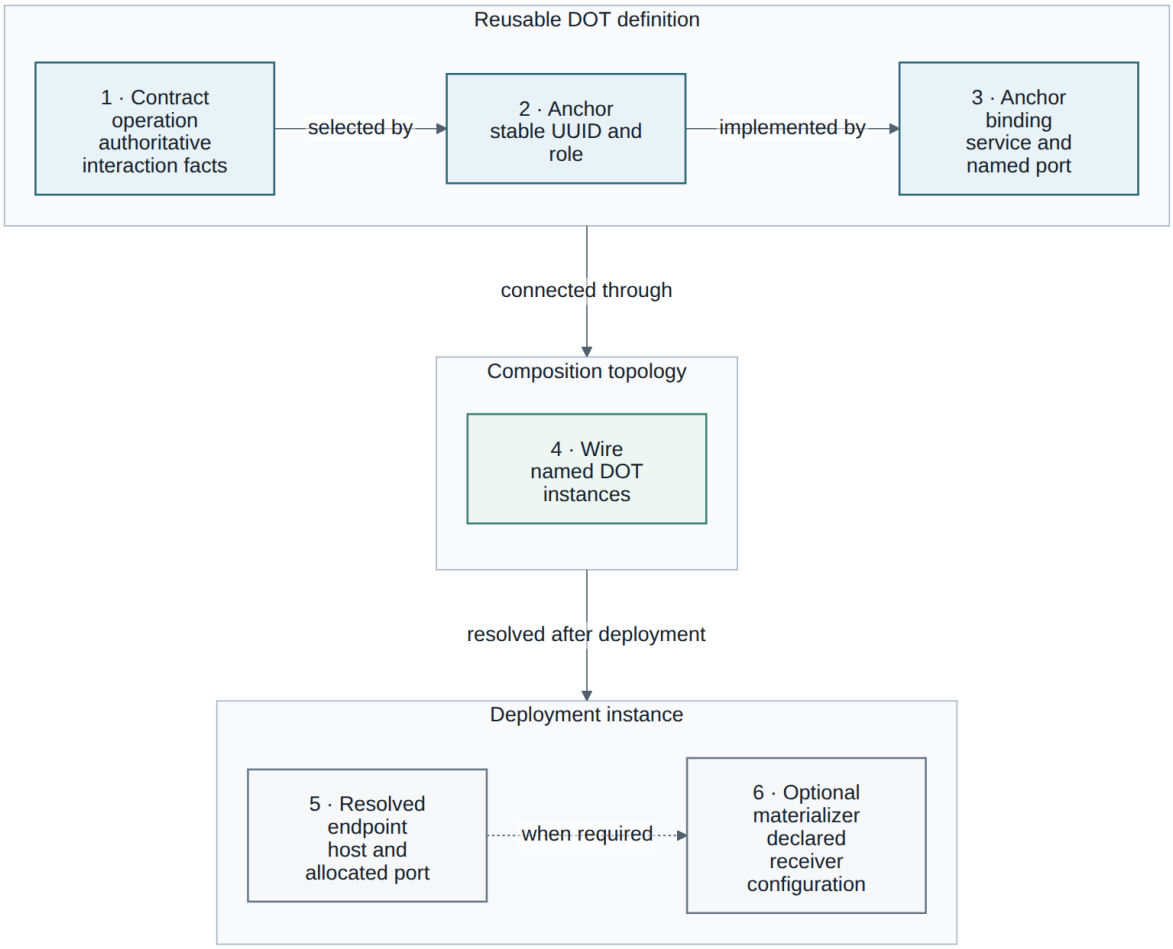


Figure 2. From an authoritative Contract operation to deployed configuration

- The **Contract** defines the interaction.
- The **Anchor** identifies one logical participation point.
- The **binding** says which declared service port implements that Anchor.
- The **wire** connects named deployment instances.
- The optional **materializer** writes a resolved value into configuration that the receiving service has already declared.

See DOT Anchors and Composition for the field model, compatibility rules, and examples.

Runtime and deployment separation

The model separates two orthogonal questions:

Concept	Field	Answers
`PlatformEngine`	`Runtime.platform.engine`	Where does the workload run? For example Docker, Kubernetes, Vagrant, or WebAssembly.
`DeploymentManifestType`	`Deployment.manifest.type`	How is it deployed? For example Docker image, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every manifest type declares its target platform. Runtime validation rejects a deployment whose manifest mechanism does not align with its platform.

`DeploymentService` is the unit that binds a deployed service to Contracts, named ports, Anchors, and supported configuration. Configuration is service-scoped; there is no deployment-level shared configuration surface.

Contract resolution

The Registry resolves an Anchor selector into a normalized Contract element:

```
AnchorContract(id, selector)
        |
        v
load Contract by UUID
        |
        v
standard-specific resolver
        |
        v
identifier + interaction + protocols + media types
          + input/output schema references + security scheme names
```

Resolution is read-only. Derived facts are returned to consumers and never written back into the Anchor. Contractless Anchors already carry their explicit direction, protocols, and media types and therefore have no selected Contract element.

Composition resolution

The provisional composition executor uses a compile-like pipeline:

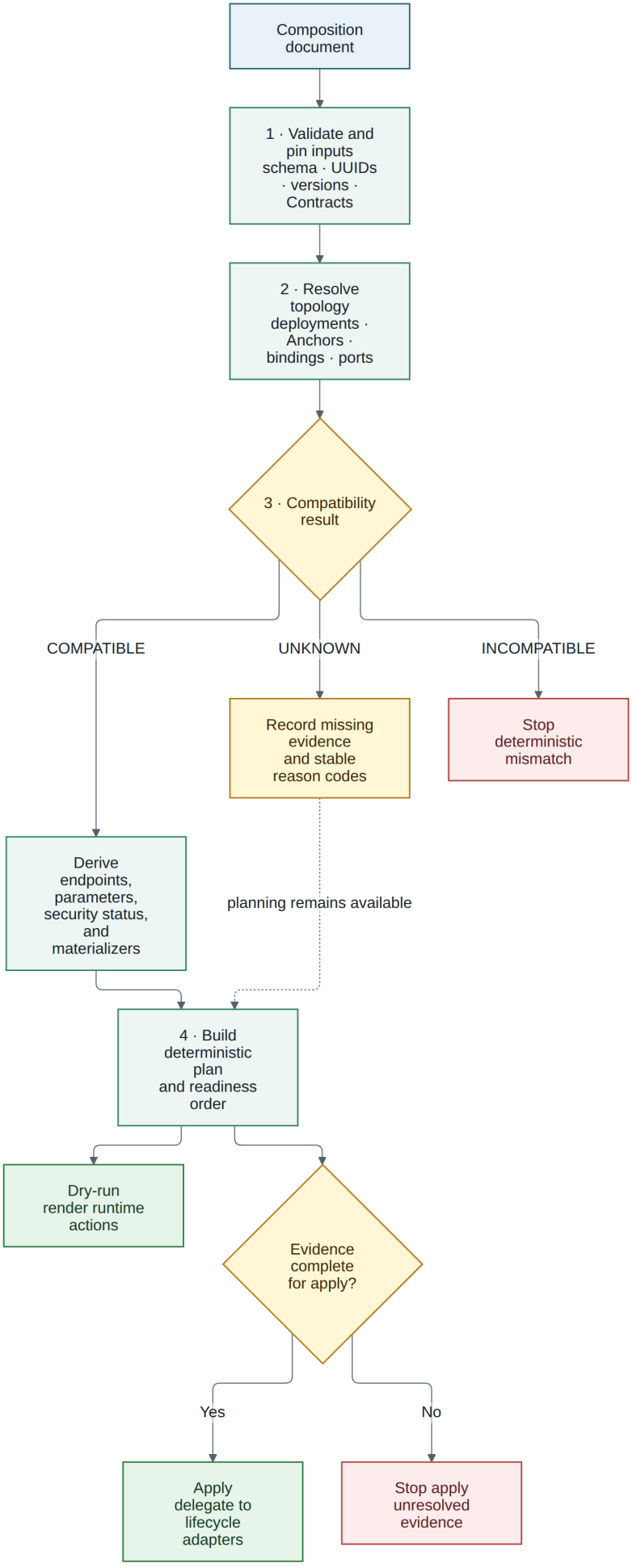


Figure 3. Fail-closed composition resolution pipeline

1. Validate the strict Draft 2020-12 composition shape.
2. Load each local DOT source and verify the pinned UUID and version.
3. Load each Contract input and verify its UUID.
4. Select one deployment for every named DOT instance.
5. Resolve Anchor references and service-port bindings.
6. Evaluate three-state compatibility: `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`.
7. Derive connection endpoints, parameter values, security status, and materializers.
8. Build a deterministic plan and readiness order.
9. On apply, prepare configuration and delegate workload operations to the existing single-DOT lifecycle adapters.

Planning may retain `UNKNOWN` with reason codes so missing evidence is visible. Apply rejects `UNKNOWN` rather than treating absence of evidence as compatibility.

Security boundaries

DOT-Design models declarations and explicit grants; it does not infer trust.

- Contract security requirements remain in the Contract.
- Sensitive deployment values are supplied by reference, not embedded in a composition or persisted as resolved values.
- OCI capabilities, host namespaces, volumes, and CDI devices require exact DOT declarations and exact operator grants.
- A configured wire does not prove credential validity, authorization, remote attestation, policy compliance, or network reachability.
- Cross-domain admission decisions belong to deployment or policy systems.

Active traffic and trust evidence must be collected by scenario-specific probes after deployment.

Persistence and API boundaries

`DOT-Registry` stores root documents in MongoDB and uses standard UUID representation. It exposes read-only Contract and Anchor endpoints, including normalized resolved Anchors. The compatibility evaluator is internal until a stable public request identity and policy boundary are defined.

`dotx compose` stores private, local lifecycle state under the user's cache directory. Per-composition locking prevents concurrent local lifecycle commands from racing that state. The current model is therefore explicitly single-host; shared orchestration state remains future work.

Design principles

- Reuse established Contract standards instead of inventing parallel protocol models.

- Keep stable logical identity separate from runtime location.
- Make adapters, transformations, privileges, and secret access explicit.
- Fail closed on ambiguity and missing security evidence.
- Preserve independently versioned DOT definitions; keep topology in composition.
- Treat generated plans and test evidence according to what they actually prove.

DOT Anchors and Composition

This guide explains how independently authored Dynamic Open Trust (DOT) components participate in composition by referring to authoritative Contracts rather than duplicating contract-defined facts. It distinguishes the stable DOT Anchor model from the provisional `dotx compose` reference format and follows the complete path from a logical capability to a deployed connection.

For the exact Registry response and error contract, see Security Function Composer handoff.

Read this first



Keep these three boundaries intact:

1. A **Contract** describes what can be exchanged.
2. An **Anchor** gives one Contract operation a stable DOT-local identity and role.
3. A **wire** connects deployed DOT instances and remains outside both DOTs.

If a value changes between installations—for example a host, URL, port, credential, realm, broker, namespace, or device—it does not belong in an Anchor.

If you want to…	Start with
Publish a composable DOT	Authoring two composable DOTs, then Contract-backed Anchors and deployment binding.
Connect existing DOTs	Composition documents and DOT instances, direct request/response wires, and materializers.
Model messaging through a broker	Brokered wires and path and channel parameters.
Handle authentication or secrets	Security and secrets.
Operate a composition	Validation, planning, and lifecycle and troubleshooting.
Design a new 6G integration type	Anchor boundaries, extension points, and 6G security composition patterns.

Release status and supported profile

The DOT-owned Anchor model, contract selectors, service-port bindings, validation, generated schema, Registry resolution, and conformance fixtures are release candidates. The separate composition document and its local lifecycle executor remain `dot.mare6g.eu/v1alpha1` reference functionality.

The current executable profile supports:

- local DOT sources pinned by UUID and version, and local Contract payloads pinned by UUID, in the composition;
- OpenAPI, AsyncAPI, Protobuf, and GraphQL contract selectors;

- contractless protocol/media declarations when no supported contract exists;
- named deployment selection and service-port bindings;
- explicit environment-variable and config-file materializers;
- Docker Image composition networking and per-instance identity;
- Kubernetes and Helm deployment/readiness through the existing single-DOT lifecycle;
- reference-only process-environment and Kubernetes Secret inputs;
- rollback, private local state, resumable partial apply, status, and reverse-order destroy; and
- fail-closed handling of ambiguity, unknown compatibility, unsupported privilege grants, and unreachable cross-platform materialization.

It is not yet a general multi-host production orchestrator. Registry/Git source resolution, shared state, cross-platform network bindings, repeated-instance isolation for every deployment mechanism, external secret-store plugins, and mutating, multi-scheme, event-driven, or cross-platform probe adapters remain outside this profile. Safe anonymous OpenAPI probes and single-scheme OAuth 2.0 client-credentials probes within a shared Docker network or Kubernetes namespace are supported.

The five distinct objects

Object	Owner	Responsibility
Contract	DOT author	Authoritative operation, message, payload, protocol, and security description in OpenAPI, AsyncAPI, Protobuf, or GraphQL.
Anchor	DOT author	Stable UUID and composition role selecting one contract element, or declaring explicit communication facts when no contract exists.
Anchor binding	DOT author	Maps one Anchor UUID to a named port of a deployment service. It does not contain a live address.
Wire	Composer/operator	Connects one named DOT instance and Anchor UUID to another. It may select a provider, consumer, or broker service-port endpoint.
Materializer	Composer/operator	Explicitly writes a resolved connection value into configuration declared by a participating service.

Where each fact lives

Fact	Source of truth	Referenced by
API operation, message, schemas, protocol, media type, security requirement	Contract payload	`Anchor.contract.selector`

Fact	Source of truth	Referenced by
Logical composition point and provider/consumer role	`Dot.anchors[]`	Wire `from` or `to`
Service and named port implementing an Anchor	`DeploymentService.anchorBindings[]`	Resolved plan endpoint
Configuration the service permits a deployer to supply	`DeploymentService.config`	Deployment input or materializer target
Selected DOT version, topology, readiness, parameter values, and secret references	Composition document	`dotx compose` plan and lifecycle
Concrete runtime address and port	Resolved deployment plan	Materializer or exposed-Anchor forwarding
Live reachability, authentication result, and policy decision	Runtime probe or external policy system	Operational evidence; never persisted as an Anchor fact

The ownership chain is:

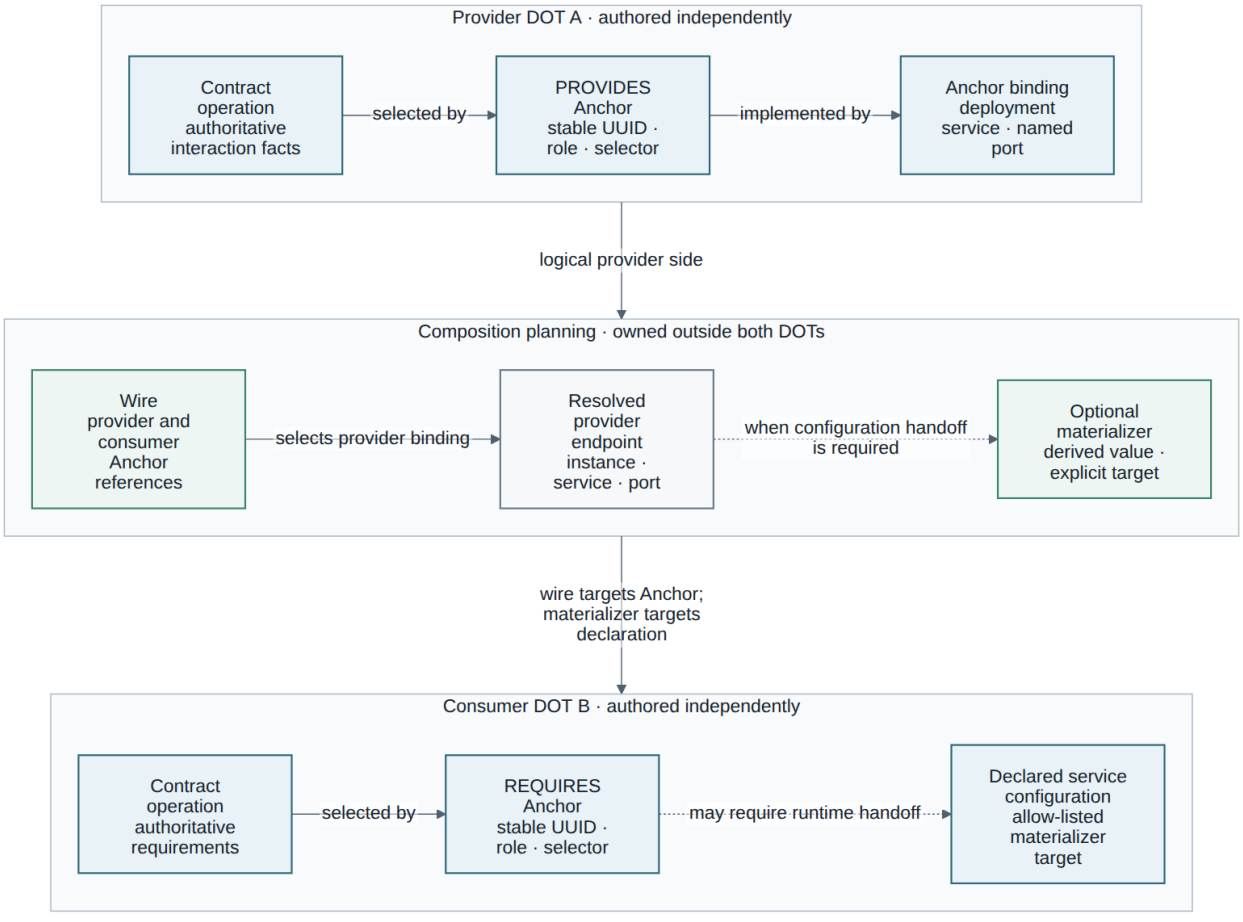


Figure 4. Provider-to-consumer ownership chain

These objects must not collapse into one schema. In particular, a wire is not part of either DOT, and a live URL is not part of an Anchor.

Resolution pipeline

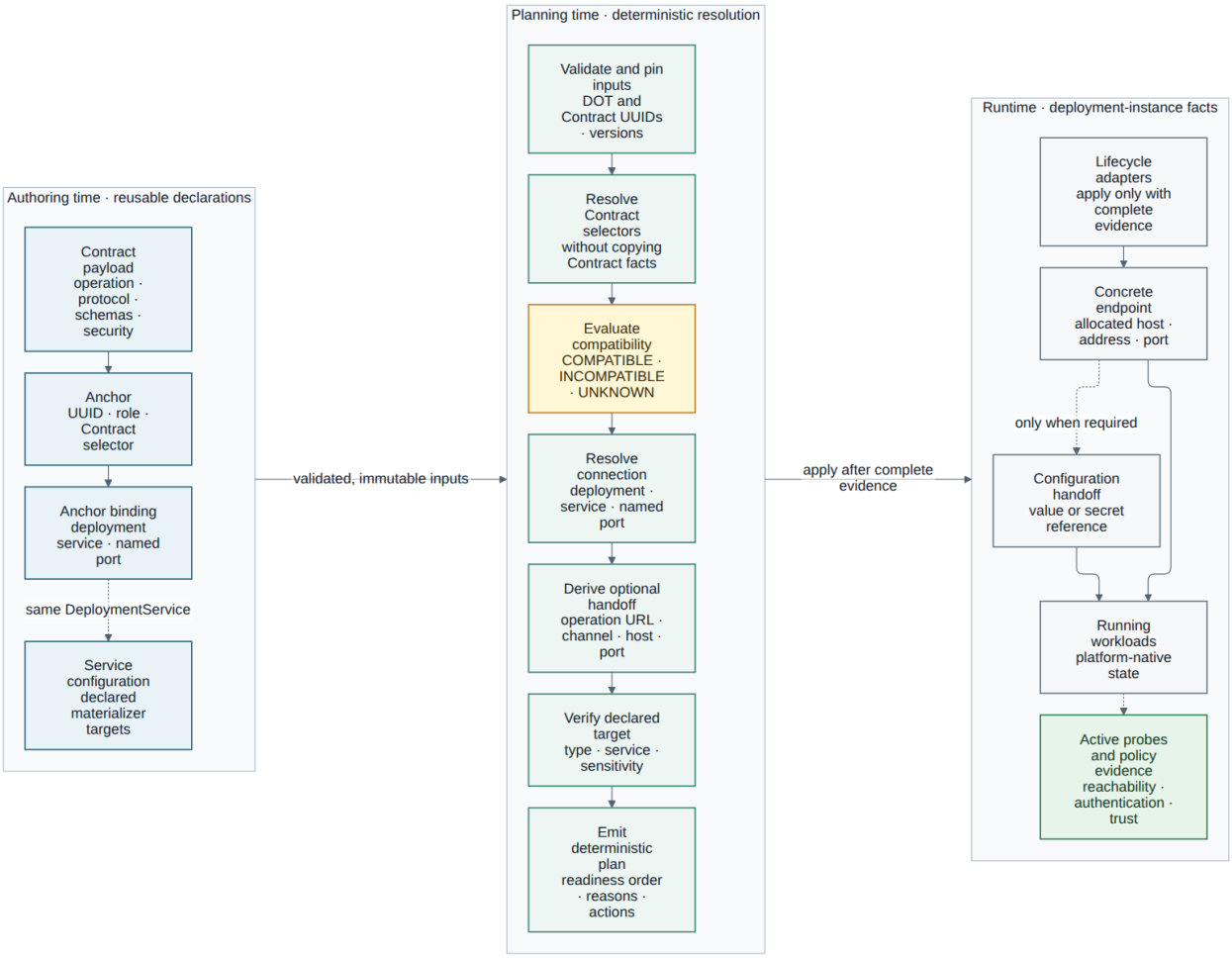


Figure 5. Authoring, planning, and runtime resolution

This resembles compilation: validate the independently authored inputs, link stable references, produce a deterministic deployment plan, and only then mutate runtime state. A failure at one phase is not evidence that a later phase was attempted.

Continue through the workflow

The detailed guide is organized around the work being performed:

Task	Guide
Publish stable composition points in a DOT	Authoring Anchors
Define pinned DOT instances and composition topology	Composition documents and instances
Connect endpoints, provide security inputs, and materialize configuration	Wiring, materialization, and security

Task	Guide
Validate, plan, apply, inspect, and evolve a composition	Composition lifecycle and design guidance

Readers implementing a separate composer should also consult the Registry handoff specification.

Authoring Anchors

This page covers the DOT publisher's side of composition. Start with the model overview if the distinction between Contracts, Anchors, bindings, and wires is not yet familiar.

Authoring two composable DOTs

Use this sequence when introducing a new connection:

1. Put the operation or message boundary in an OpenAPI, AsyncAPI, Protobuf, or GraphQL Contract. Use the contractless form only when none of those standards can describe the boundary.
2. Add a stable `PROVIDES` Anchor to the provider DOT and a stable `REQUIRES` Anchor to the consumer DOT. Select the authoritative Contract element from each Anchor; do not copy its protocol or payload fields.
3. On every deployable implementation, bind each Anchor UUID to the named `ServicePort` that implements it.
4. Declare every consumer configuration input on its `DeploymentService`, including sensitivity. This declaration is the allow-list for composition.
5. Validate each DOT independently with `dotx validate` before writing a composition.
6. Add named, UUID/version-pinned DOT instances and UUID-pinned Contract inputs to a separate composition document.
7. Connect the Anchor UUIDs with a wire. Add an explicit broker endpoint, parameter values, security binding, or materializer only when the connection requires it.
8. Run `dotx compose lock` and commit the generated lock beside the composition. Review digest changes as changes to the resolved DOT-to-DOT graph.
9. Run `dotx compose validate` to check the document and `dotx compose plan` to inspect the resolved topology. Treat `UNKNOWN` compatibility as missing evidence, not as success.
10. Use `apply --dry-run` before execution. After apply, use `compose probe` for safe Contract-derived traffic and `status` for native runtime status, configuration evidence, and the latest redacted probe evidence. Dedicated acceptance tests remain necessary for authentication, authorization, mutable operations, and application-specific outcomes.

The complete three-DOT worked example is under `DOT-Testing/e2e/api-gateway-oidc/`: its `keycloak/`, `apisix/`, and `oauth-client/` directories contain independently valid DOTs; `contracts/` contains the authoritative APIs; and `scenario.yaml` owns the wiring.



Concrete identifiers in this guide use the canonical lowercase UUID text form with

> UUIDv4 version and IETF variant bits defined by RFC 9562. Deterministic values are retained where they identify executable repository fixtures, making the documentation traceable to the tested examples. Generate new UUIDs for production DOTs; do not reuse documentation or fixture identifiers.

Contract-backed Anchors

A contract-backed Anchor carries stable identity and selects one authoritative contract element. It deliberately omits protocol, direction, media type, payload schema, and security fields that the Contract already defines.

```
{
  "id": "a0000000-0000-4000-8000-000000000001",
  "name": "submit-alert-http",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:alert"],
  "contract": {
    "id": "11111111-1111-4111-8111-111111111111",
    "selector": "#/paths/~1alerts/post"
  }
}
```

The opposite DOT declares its own `REQUIRES` Anchor. It may select the same Contract UUID or an independently authored Contract. Compatibility is deterministic when authoritative facts are comparable; otherwise the result is `UNKNOWN`, never an invented match.

Reading compatibility results

The plan compares the provider and consumer across independently explainable dimensions:

Dimension	Meaning
Role and direction	The connection is from `PROVIDES` to `REQUIRES`, and the selected interaction directions can communicate.
Semantics	Declared semantic-type URIs intersect when both sides provide them.
Protocol and media	Resolved protocol and media-type sets have a usable intersection.
Interaction	Request/response, publish/subscribe, streaming, query, mutation, or subscription behavior agrees.
Schema	Provider output and consumer input schema identities are compared in the correct direction when both select the same authoritative Contract; cross-Contract structural comparison remains `UNKNOWN`.
Security	Contract requirements are understood and, for execution, a complete supported binding has all required declared inputs.

`INCOMPATIBLE` means comparable evidence conflicts. `UNKNOWN` means evidence is missing or the current resolver cannot prove the dimension. Validate and plan may explain `UNKNOWN`; apply rejects

it. An explicit mediator DOT is preferable to weakening compatibility when protocols or payloads genuinely differ.

Selector syntax

Standard	Selector	Example
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/paths/~1alerts/post`
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`#/operations/publishAlert`
Protobuf	Fully qualified service method	`mare.security.AlertService.PublishAlert`
GraphQL	Root-field schema coordinate	`Mutation.publishAlert`

For OpenAPI and AsyncAPI, `/` in a JSON Pointer token becomes `~1` and `~` becomes `~0`. Concrete path or channel template values belong to a composition wire's `parameters`, not to the Anchor.

Supported Contract representation matrix

Standard	Contract `format`	Authoritative interaction	Composition evidence
OpenAPI	`YAML` or `JSON`	Request/response operation, including path and security requirements	One shared selector; `REQUEST_RESPONSE` for both participants
AsyncAPI	`YAML` or `JSON`	Message operation and Channel Object	Distinct complementary selectors; `SEND` and `RECEIVE`, plus the resolved channel address
Protobuf / gRPC	`PROTOBUF`	Fully qualified RPC and unary or streaming shape	One shared method selector; `REQUEST_RESPONSE`, `CLIENT_STREAM`, `SERVER_STREAM`, or `BIDIRECTIONAL_STREAM`
GraphQL	`GRAPHQL`	SDL root field	One shared schema coordinate; `QUERY`, `MUTATION`, or `SUBSCRIPTION`

`dotx compose plan --json` exposes the resolved `contractStandard`, `contractFormat`, `providerInteraction`, and `consumerInteraction` on each contract-backed wire. These fields are derived from the supplied Contract and therefore do not duplicate authoring metadata in either Anchor or composition. The deterministic `DOT-Testing/composition/contract-standards/` matrix connects one provider DOT to one consumer DOT through every row above.

Semantic types

`semanticTypes` is a list of URI identifiers describing the security or domain meaning of the interaction independently of its transport. Prefer stable, governed ontology or vocabulary URIs when one exists; a project URN is suitable for concepts that have no external vocabulary yet.

Current matching is deliberately conservative: both Anchors must declare at least one semantic type and their sets must intersect exactly. Missing values produce `UNKNOWN`; different values produce `INCOMPATIBLE`. The evaluator does not infer subclass, equivalence, or broader/narrower ontology relations. Such reasoning belongs in a separately reviewed semantic-policy layer.

Contractless Anchors

Use a contractless Anchor only when no supported authoritative contract exists, for example a raw packet stream or a legacy metrics boundary. It must state the facts that cannot otherwise be derived:

```
{
  "id": "a0000000-0000-4000-8000-000000000006",
  "name": "packet-stream",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:packet-stream"],
  "direction": "EGRESS",
  "protocols": ["TCP"],
  "mediaTypes": ["application/octet-stream"]
}
```

Contract-backed and contractless forms are mutually exclusive. A contract-backed Anchor must not repeat `direction`, `protocols`, or `mediaTypes`. A contractless Anchor must declare all three.

Role is not traffic direction

`role` answers who offers the selected capability:

- `PROVIDES` means this DOT makes the capability available;
- `REQUIRES` means this DOT depends on that capability.

It does not mean that bytes move in only one direction. An OpenAPI request/response operation has a providing server and requiring client while both send data. An AsyncAPI provider may select a `send` operation and its consumer a complementary `receive` operation. For a contract-backed Anchor, the resolver derives that interaction from the selected operation. Only a contractless Anchor states `INGRESS`, `EGRESS`, or `BIDIRECTIONAL` explicitly.

When two DOTs implement the same published Contract, referencing the same Contract UUID gives the strongest deterministic evidence. Independently authored Contracts are allowed, but identical-looking schema names do not prove structural compatibility across Contract identities; that comparison remains `UNKNOWN` until a schema-aware adapter or policy evaluates it.

Stable identity and version evolution

- The UUID, not `name`, identifies an Anchor in a wire.
- A rename or documentation correction preserves the UUID.
- A compatible DOT version preserves the UUID for the same logical Anchor.

- An incompatible semantic replacement receives a new UUID.
- Removal and any future `deprecated`/`replacedBy` metadata are publishing governance decisions; they are not inferred from contract changes.
- Names remain useful in logs and displays but are never foreign keys.

Binding an Anchor to a deployment

An Anchor is logical until a selected deployment maps it to a named service port:

```
{
  "id": "security-function",
  "name": "Security Function",
  "host": "security-function",
  "ports": [
    {"name": "http", "protocol": "HTTP", "port": 8080}
  ],
  "anchorBindings": [
    {
      "anchorId": "a0000000-0000-4000-8000-000000000001",
      "port": "http"
    }
  ],
  "config": {
    "environment": [],
    "configFiles": [],
    "volumes": []
  }
}
```

This says which logical service and named port implement the Anchor. The deployer still decides the concrete container DNS name, cluster Service, allocated address, forwarded port, certificate, or network path.

`@ValidAnchor` and `AnchorValidator` reject duplicate Anchor UUIDs, undeclared Contract references, invalid selector syntax, undeclared ports, undeclared Anchor bindings, and duplicate service-local bindings.

Runtime configuration portability

Every `DeploymentService` declares its supported configuration surface through `environment`, `configFiles`, and logical `volumes`. A composition may supply only values whose names or target paths the selected service declares. Required environment variables and files without defaults fail before runtime mutation; sensitive environment values require a `secretRef` instead of a literal. When an input is omitted, `dotx` reuses the non-sensitive `defaultValue` or `defaultContent` declared by that service. Explicit composition values and wire materializers override defaults; conflicting explicit inputs fail closed. Declarations marked `sensitive` are forbidden from embedding either kind of default and must use a deployment-time secret reference or runtime-native secret facility.

Each composition input carries the stable `service` id it targets. Authors may omit `service` only when exactly one service in the selected deployment declares that environment name, file path, or volume name; planning resolves and records the service explicitly. If two services declare the same item, omission is an error. Duplicate detection is service-local, so two services can intentionally receive different values for the same environment-variable name.

Environment variables and configuration files are rendered by every currently executable `dotx` deployment adapter:

Manifest	Environment	Configuration files
`DOCKER_IMAGE`	Container environment	Read-only bind mounts
`DOCKER_COMPOSE`	Per-service override using process-environment staging	Generated per-service Compose override mounts
`KUBERNETES_MANIFEST`	Exact workload/container patch; existing Secret keys remain `secretKeyRef`	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`HELM_CHART`	Exact post-install workload/container patch; existing Secret keys remain native	Service-qualified ConfigMap or Secret plus exact workload/mount patch
`VAGRANTFILE`	Vagrant process/provisioner environment	Files provisioned into the managed VM and exposed to the Vagrantfile

Configuration remains service-scoped. A Docker image or Vagrant deployment is one workload or VM, so several logical services may safely share its declared configuration surface. Docker Compose, Kubernetes, and Helm may materialize several independent workloads. Their composition therefore supplies `deploymentInputs.workloads`: a Compose binding names one native service; a Kubernetes/Helm binding supplies exact native labels and optionally a container. Dotx intersects those labels with its composition-owned instance label and requires exactly one workload. Missing, duplicate, cross-runtime, ambiguous workload, and ambiguous container bindings fail closed. This prevents environment values, secrets, or mounted files from reaching the wrong security function without adding platform-specific selectors to the reusable DOT model.

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Storage and privilege configuration is deliberately runtime-native. Docker image deployments support composition-scoped named volumes, explicit OCI bind sources, exact capability/host-namespace grants, and CNCF CDI devices. Compose payloads own their volume graph; Kubernetes

and Helm own claims, storage classes, and volume topology; Vagrantfiles own disks and synced folders. A composer must not translate a host bind path into a cluster volume or silently grant privileged mode.

`WASM_MODULE` is a domain extension point, not an executable `dotx` adapter yet. It remains fail-closed until a reviewed WASI component/runtime profile defines environment, preopened directories, networking, secrets, and lifecycle semantics. The deterministic `DOT-Testing/composition/runtime-configuration/` matrix exercises the same catalogued DOT through every executable adapter. The `DOT-Testing/composition/service-workloads/` matrix verifies isolation between two services across Compose, Kubernetes, and Helm dry-run adapters.

Discovering Anchors through the Registry

A composer that obtains DOTs from the Registry should prefer resolved Anchors:

```
GET /dots/\{dotId}/anchors
GET /dots/\{dotId}/anchors/\{anchorId}
GET /dots/\{dotId}/anchors/resolved
GET /dots/\{dotId}/anchors/\{anchorId}/resolved
GET /contracts/\{contractId}
```

The unresolved endpoints return the DOT-authored Anchor. A resolved response adds versioned Contract metadata and the normalized selected element:

```
{
  "apiVersion": "1",
  "anchor": {
    "id": "a0000000-0000-4000-8000-000000000001",
    "name": "submit-alert-http",
    "role": "PROVIDES",
    "semanticTypes": ["urn:mare:security:alert"],
    "contract": {
      "id": "11111111-1111-4111-8111-111111111111",
      "selector": "#/paths/~1alerts/post"
    }
  },
  "contract": {
    "id": "11111111-1111-4111-8111-111111111111",
    "name": "alert-http-api",
    "version": "1.0.0",
    "standard": "OPENAPI",
    "format": "YAML"
  },
  "selectedElement": {
    "selector": "#/paths/~1alerts/post",
    "identifier": "submitAlert",
    "interaction": "REQUEST_RESPONSE",
    "protocols": ["HTTPS"],
    "mediaTypes": ["application/json"],
```

```
    "inputSchemas": ["#/components/schemas/Alert"],
    "outputSchemas": ["#/components/schemas/Receipt"],
    "securitySchemes": ["oauth2"]
  }
}
```

Resolved responses omit the potentially large Contract payload. Fetch it from `/contracts/{contractId}` only when the composer must inspect the authoritative document. For a contractless Anchor, `contract` and `selectedElement` are `null`. A dangling Contract reference is an error, not a contractless Anchor.

The compatibility evaluator is not currently a public Registry endpoint. `dotx compose` performs the equivalent conservative evaluation for its local sources; another composer should use the normalized resolved facts and the three-state rules described above rather than treating missing evidence as a match. See the SFC handoff for the complete response and error contract.

Next, define the concrete participants and topology in a composition document.

Composition Documents and DOT Instances

A composition document is owned by the composer or operator, not by any member DOT. It selects immutable DOT identities and creates deployment instances whose topology may vary between environments.

Composition documents and DOT instances

A composition is a separate versioned document. It names concrete instances of DOT definitions and pins their identities through the authoritative `DOT-Repository` catalogue. The `repository://<dot-id>@<version>` form is stable across checkout locations and prevents a scenario-local copy from drifting away from the reviewed definition:

```
apiVersion: dot.mare6g.eu/v1alpha1
kind: Composition
id: e3000000-0000-4000-8000-000000000001
name: api-gateway-oidc
version: 1.0.0
dots:
  - name: identity
    source: repository://d3000000-0000-4000-8000-000000000001@1.0.0
    dotId: d3000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: kubernetes
  - name: gateway
    source: repository://d3a00000-0000-4000-8000-000000000002@1.0.0
    dotId: d3a00000-0000-4000-8000-000000000002
    dotVersion: 1.0.0
    deployment: kubernetes
    dependsOn: [identity]
contracts:
```

```
  - id: c3000000-0000-4000-8000-000000000001
    source: ./contracts/oidc-discovery.yaml
```

`name` identifies an instance inside this composition. `source` selects the catalogued definition by UUID and exact version. `dotx compose` locates the nearest `DOT-Repository/dot-repository.yaml`, resolves the entry to its reviewed bundle, and then verifies that `dotId` and `dotVersion` match the loaded `dot.json`. Neither a catalogue entry nor a local path can silently substitute a different DOT. `deployment` selects a deployment by its stable deployment ID or unambiguous manifest alias.

Relative paths remain supported for authoring experiments and external Contract documents. Reusable stable, sample, and test-only DOT definitions belong in `DOT-Repository`, under their corresponding catalogue scope. Contracts remain authoritative protocol documents and are referenced independently; placing a DOT in the repository does not copy OpenAPI, AsyncAPI, Protobuf, or GraphQL facts into its Anchors.

`dependsOn` is a readiness graph, not a connection graph. A wire does not imply startup order. This distinction permits reciprocal peer wires and brokered topologies without creating false dependency cycles.

Composition field reference

The emitted Draft 2020-12 JSON Schema is normative for the serialized `v1alpha1` shape. This table is the study guide for its top-level ownership:

Field	Required	Meaning
`apiVersion`, `kind`	Yes	Fixed discriminators: `dot.mare6g.eu/v1alpha1` and `Composition`.
`id`	Yes	Stable UUID of this composition, also used to derive local state and runtime identities.
`name`, `version`	Yes	Human-readable identity and version of the composition document.
`dots[]`	Yes, at least one	Named deployment instances. Every entry pins `source`, `dotId`, and `dotVersion`; published scenarios use an exact `repository://<uuid>@<version>` source. `deployment`, `dependsOn`, and `deploymentInputs` refine execution.
`contracts[]`	When a referenced Contract is needed	Maps an exact Contract UUID to a local Contract document. The document's embedded `id` must match.
`wires[]`	No	Connections between one `PROVIDES` and one `REQUIRES` Anchor. A composition with no wires can still expose a standalone capability.
`exposedAnchors[]`	No	Composition-local names for member Anchors that an operator may expose or forward.

Each wire requires `name`, `from`, and `to`. Both ends contain an instance name and Anchor UUID. Optional `endpoint` selects a third-party or otherwise non-default service port; `parameters` supplies only Contract-declared template values; `materializer` or `materializers` writes resolved values; and `security` binds Contract security requirements to declared participant configuration. Unknown fields are rejected.

Deployment inputs and native workload bindings

`deploymentInputs.environment`, `configFiles`, and `volumes` identify the logical `DeploymentService` that owns each input. `ociRuntime` and `cdiDevices` authorize only requirements already declared by that selected DOT deployment. These fields supply instance values and grants; they do not expand the DOT's declared configuration surface.

When a Docker Compose, Kubernetes, or Helm deployment contains several logical services with configuration, `deploymentInputs.workloads` is required for every configured service. The binding is deliberately instance-owned because native service names, labels, and containers can differ between deployment variants or environments:

```
deploymentInputs:
  workloads:
    - service: api
      compose:
        service: backend
```

The Kubernetes/Helm form uses exact-match standard labels and may select one container:

```
deploymentInputs:
  workloads:
    - service: api
      kubernetes:
        matchLabels:
          app.kubernetes.io/component: api
        container: api
```

Labels are combined with dotx's instance ownership selector. A live apply must resolve exactly one Deployment, StatefulSet, or DaemonSet, and an omitted container is accepted only when that workload has one container. Two logical services cannot target the same Compose service or the same Kubernetes workload/container pair. Plans expose the binding and secret identifiers, and locks cryptographically pin them; neither contains resolved secret values.

DOT definition versus DOT instance

One pinned DOT definition can appear more than once in a composition. Each entry has a distinct composition-local `name`, runtime identity, deployment inputs, and wires while retaining the same `dotId` and `dotVersion`:

```
dots:
```

```
  - name: site-a
    source: ./federation-agent
    dotId: d5000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-a}
  - name: site-b
    source: ./federation-agent
    dotId: d5000000-0000-4000-8000-000000000001
    dotVersion: 1.0.0
    deployment: docker
    deploymentInputs:
      environment:
        - {service: federation-agent, name: FEDERATION_NODE_ID, value: site-b}
```

Wires reference `site-a` or `site-b`, never the source path. Docker Image deployments receive deterministic per-instance container, DNS, network, and named-volume identities. Apply rejects repeated instances for deployment mechanisms whose manifest-owned resource names cannot yet be isolated safely.

The executable example is `DOT-Testing/composition/federation-peers/scenario.yaml`.

After defining the participants, wire and configure their selected Anchors.

Wiring, Materialization, and Security

Wires express composition topology outside every participating DOT. They refer to stable Anchor UUIDs, resolve deployable endpoints through service bindings, and materialize only values that the target service has declared as configurable.

Direct request/response wire

A wire uses instance names and stable Anchor UUIDs:

```
wires:
  - name: oauth-token-to-client
    from:
      instance: identity
      anchorId: a3000000-0000-4000-8000-000000000002
    to:
      instance: client
      anchorId: a5000000-0000-4000-8000-000000000001
    materializer:
      type: ENVIRONMENT_VARIABLE
      valueFrom: OPERATION_URL
      target:
        instance: client
```

```
        service: oauth-client
        environmentVariable: TOKEN_URL
```

Planning performs the following sequence:

1. Resolve the two pinned DOT instances.
2. Resolve both Anchor UUIDs and require complementary `PROVIDES`/`REQUIRES` roles.
3. Resolve the selected Contract elements.
4. Evaluate semantic, interaction, protocol, media, schema, and security compatibility.
5. Resolve the connection endpoint through the selected deployment's `anchorBindings`.
6. Verify that the target service declares `TOKEN_URL`.
7. Build `OPERATION_URL` from the runtime endpoint and the authoritative OpenAPI server/operation path.

The resulting URL is deployment data. It is never copied back into `dot.json` or the Anchor.

A wire and a materializer are intentionally separate. The wire says *these Anchors are connected*; the materializer says *write this derived value into this declared consumer setting*. A wire needs no materializer when the runtime already provides discovery, a service mesh performs routing, both participants use a fixed broker declared elsewhere, or the composition is documenting an Anchor relation without an executable handoff. Apply still requires every actually required service configuration value to have a default or explicit input.

For the example above, the human-readable plan makes every resolution step inspectable before apply:

```
oauth-token-to-client:
  identity/a3000000-0000-4000-8000-000000000002
    -> client/a5000000-0000-4000-8000-000000000001 [COMPATIBLE]
  endpoint: keycloak:8080
    (HTTP, instance=identity service=keycloak port=http)
  contract: c3000000-0000-4000-8000-000000000002
    selector=#/paths/~1realms~1dotx~1protocol~1openid-connect~1token/post
  materializer: OPERATION_URL -> client/oauth-client env:TOKEN_URL
```

The same plan with `--json` is the machine-readable automation output. It includes stable instance and Anchor references, compatibility reasons, the selected Contract, resolved endpoint, derived operation/channel value, materializer targets, security status, and exposed-Anchor endpoints. Consumers should parse JSON rather than scrape the human-readable form.

Brokered wire

For messaging, the semantic provider, semantic consumer, and connection endpoint may be three different instances. Select the broker explicitly:

```
wires:
  - name: openc2-command-via-mqtt
    from:
      instance: interpreter
      anchorId: a6000000-0000-4000-8000-000000000003
    to:
      instance: actuator
      anchorId: a6000000-0000-4000-8000-000000000006
    endpoint:
      instance: broker
      service: mqtt
      port: mqtt
    materializers:
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_HOST
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_HOST
      - type: ENVIRONMENT_VARIABLE
        valueFrom: ENDPOINT_PORT
        target:
          instance: actuator
          service: openc2-actuator
          environmentVariable: MQTT_BROKER_PORT
```

The AsyncAPI operations and message schema stay in the Contract. The broker's host and port stay in the resolved plan. If a channel has a templated address, the wire supplies only values for parameters declared by the Contract.

Materializers

Materializers are explicit adapters between a resolved endpoint and a configuration surface already declared by a service.

Type	Behavior
`ENVIRONMENT_VARIABLE`	Writes one resolved scalar to a declared environment variable.
`CONFIG_FILE`	Mounts a declared file and optionally replaces one YAML/JSON value selected by an RFC 6901 JSON Pointer.

Supported `valueFrom` values are:

Value	Result
`ENDPOINT_HOST`	Resolved service host only.
`ENDPOINT_PORT`	Resolved numeric service port only.
`ENDPOINT_HOST_PORT`	`host:port`.

Value	Result
`ENDPOINT_URL`	`scheme://host:port`, using the bound port protocol.
`OPERATION_URL`	Endpoint URL plus the authoritative OpenAPI server base and operation path.
`CHANNEL_ADDRESS`	Address of the authoritative AsyncAPI Channel Object.

An optional RFC 6570 `uriTemplate` can derive an application-specific URI from one resolved scalar. It is explicit composition behavior, not an Anchor or Contract fact.

Configuration-file materialization

A `CONFIG_FILE` materializer always names an operator-controlled source and a file path already declared by the target service. Without `valueFrom`, the source is supplied unchanged. With `valueFrom`, the source must be YAML or JSON and `jsonPointer` selects the scalar to replace:

```
materializer:
  type: CONFIG_FILE
  source: ./prometheus/prometheus.yml
  valueFrom: ENDPOINT_HOST_PORT
  target:
    instance: prometheus
    service: prometheus
    path: /etc/prometheus/prometheus.yml
    jsonPointer: /scrape_configs/0/static_configs/0/targets/0
```

This produces a deployable Prometheus configuration without putting the provider's runtime address in either DOT. The corresponding executable fixture is `DOT-Testing/composition/poc4-continuous-monitoring/scenario.yaml`.

URI-template derivation

Use `uriTemplate` only when the application needs an explicit deterministic derivation that is not already the selected Contract operation URL:

```
materializer:
  type: ENVIRONMENT_VARIABLE
  valueFrom: ENDPOINT_URL
  uriTemplate: "{+value}/keycloak/realms/drax"
  target:
    instance: proxy
    service: proxy-xapp
    environmentVariable: OIDC_ISSUER
```

The RFC 6570 variable is named `value`. The template belongs to the composition because it adapts one deployment to another; it does not amend the Contract or redefine the Anchor.

Materialization fails when the target configuration is undeclared, the Contract path/channel

remains unresolved, alternative server paths are ambiguous, two wires produce conflicting values for one target, or the endpoint and target are on different runtime platforms without an explicit network binding.

Path and channel parameters

Only deployment-specific scalar values belong in the wire:

```
parameters:
  realm:
    value: drax
```

The Contract remains authoritative for the parameter name, location, requirement, schema, default, and serialization rules. Current composition bindings cover OpenAPI path parameters and AsyncAPI channel parameters. Query/header/cookie parameters, complex serialization, and runtime-derived parameter values require explicit future adapters.

Security and secrets

Contract security remains authoritative. A wire may demonstrate how one declared scheme is satisfied by deployment configuration:

```
security:
  bindings:
    - scheme: bearerAuth
      mechanism: https://www.rfc-editor.org/rfc/rfc6749#section-4.4
      participant: {instance: client, service: oauth-client}
      parameters:
        tokenUrl: {environmentVariable: TOKEN_URL}
        clientId: {environmentVariable: CLIENT_ID}
        clientSecret: {environmentVariable: CLIENT_SECRET}
```

Sensitive inputs are references, never secret values. They are scoped to the DOT instance whose selected deployment consumes them:

```
dots:
  - name: client
    source: repository://d3a00000-0000-4000-8000-000000000003@1.0.0
    dotId: d3a00000-0000-4000-8000-000000000003
    dotVersion: 1.0.0
    deployment: kubernetes
    deploymentInputs:
      environment:
        - name: CLIENT_SECRET
          secretRef:
            store: kubernetes
            name: oauth-client
```

```
            key: client-secret
```

Plans and composition state keep only the reference. Docker deployments can resolve `store: environment` at final handoff; Kubernetes and Helm preserve `store: kubernetes` as `secretKeyRef` without reading the Secret. A `CONFIGURED` plan proves configuration completeness, not token validity, issuer trust, mTLS, workload identity, or remote attestation.

Exposed Anchors

A composition may re-export a member Anchor under a composition-local name:

```
exposedAnchors:
  - name: protected-metadata-through-gateway
    target:
      instance: gateway
      anchorId: a4000000-0000-4000-8000-000000000003
```

At apply time, an operator may request a local forward:

```
dotx compose apply scenario.yaml \
  --forward protected-metadata-through-gateway=19080
```

The exposure name belongs to the composition boundary. It does not rename or replace the member Anchor UUID.

Before runtime mutation, follow the validation and lifecycle workflow.

Composition Lifecycle and Design Guidance

The lifecycle deliberately separates reproducible input locking, model validation, deterministic planning, runtime mutation, recorded state, and active acceptance evidence. This keeps review and CI read-only while making execution recoverable and auditable.

Validation, planning, and lifecycle

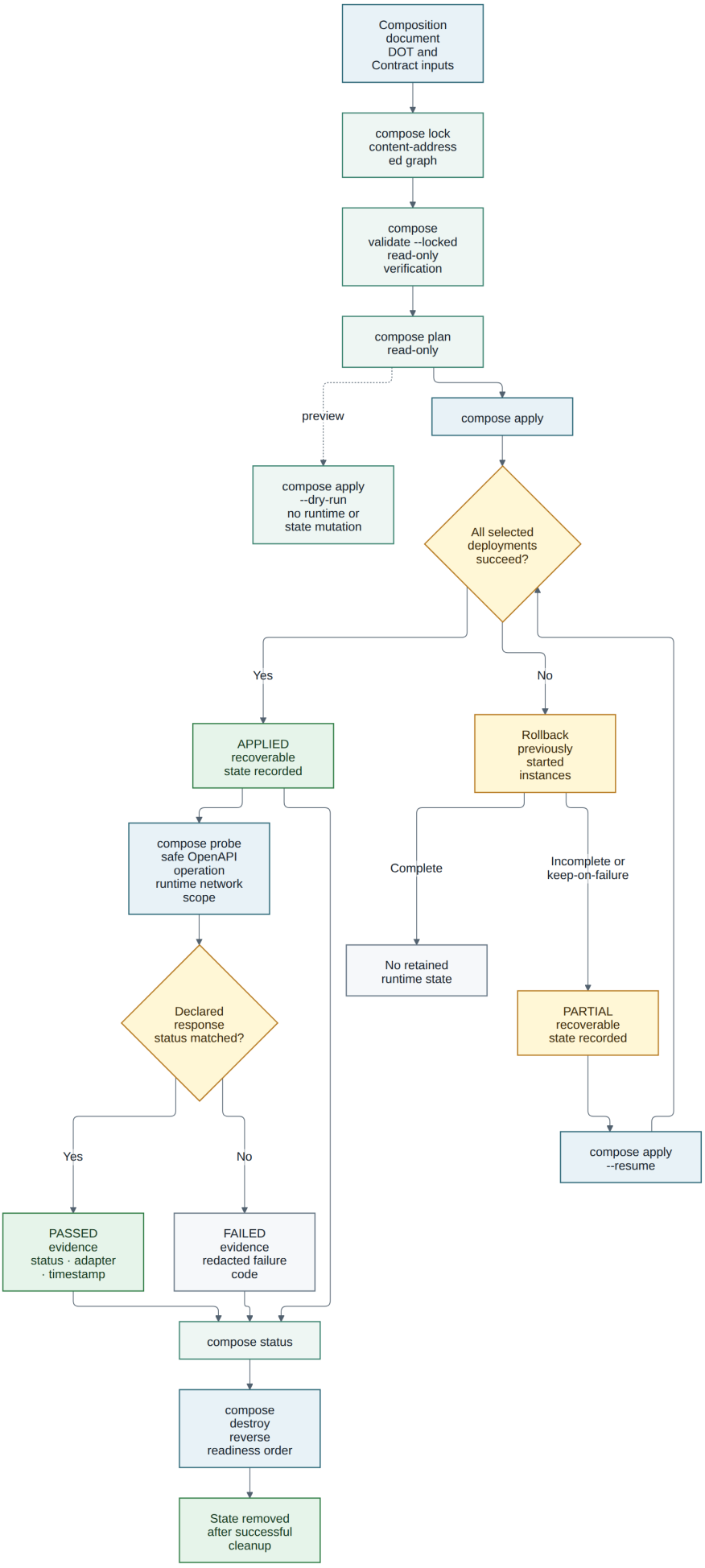


Figure 6. Composition lifecycle

What each command proves

Command	Proves	Does not prove	Mutates runtime/state
`compose lock`	The exact composition, selected DOT definitions and deployments, Contracts, configuration inputs, and portable resolved plan are content-addressed.	Runtime availability, credentials, or policy acceptance.	Writes only the requested lock file; `--check` is read-only
`compose validate`	The strict document shape, references, selected deployments, Anchors, Contracts, compatibility evidence, and materializer declarations resolve.	The exact runtime actions or live availability.	No
`compose plan`	Everything validated above plus the deterministic instance order, runtime identities, endpoints, Contract standard/format and participant interaction, derived values, security status, and materializers.	That the planned hosts are reachable or credentials work.	No
`compose apply --dry-run`	The composition reaches lifecycle preparation and can render supported deployment actions without persisting composition state.	Successful deployment or application-level traffic.	No
`compose apply`	The supported deployment adapters completed and recoverable state was recorded, subject to rollback behavior.	End-to-end Contract behavior unless separately probed.	Yes
`compose probe`	A selected safe OpenAPI operation is reachable from the composition's shared Docker network or Kubernetes namespace. Anonymous results match a declared response; supported OAuth results must match a declared `2xx` response.	Response-body conformance, mutating or multi-scheme operations, authorization-policy quality, AsyncAPI delivery, or full application semantics.	Creates and removes an ephemeral probe runner without changing member workloads; writes redacted probe evidence to composition state

Command	Proves	Does not prove	Mutates runtime/state
`compose status`	Native status for recorded instances, wire/exposure configuration evidence, and the latest recorded active-probe outcomes.	Current reachability after the probe timestamp, authentication mechanisms not explicitly identified by the evidence, authorization semantics, response bodies, or policy compliance.	No runtime mutation; reads state while holding the lifecycle lock
`compose destroy`	Owned instances were destroyed in reverse order and successful cleanup removed composition state.	Cleanup of resources not owned or recorded by this composition.	Yes

`validate` answers *is this composition internally resolvable?* `plan` answers *what exactly would be connected and materialized?* Keeping both commands read-only makes them suitable for review and CI.

`lock` makes those answers reproducible. Its JSON output preserves the original `repository://` and relative references while recording SHA-256 digests for the composition, every resolved `dot.json`, every supplied Contract, configuration inputs, and the portable resolved plan. It never records secret values, runtime addresses, or checkout-local absolute paths. Validate, plan, and apply verify an adjacent lock automatically; `--locked` additionally requires it, and `--lock <path>` selects an explicit file. Any mismatch fails before runtime mutation.

`probe` deliberately derives its request from the selected OpenAPI operation instead of introducing health-check fields in an Anchor. It accepts only `GET`, `HEAD`, and `OPTIONS`, requires one fully bound path, and executes a digest-pinned curl runner inside the common Docker network or Kubernetes namespace. The OAuth 2.0 adapter reuses one configured RFC 6749 client-credentials binding that satisfies a complete, single-scheme OpenAPI Security Requirement alternative. It resolves the provider-derived token URL, client ID, scopes, and client-secret reference at execution time, then requires a Contract-declared `2xx` response from the protected operation. Kubernetes Secret values remain `secretKeyRef` references; Docker secrets are supplied only through the runner environment. Neither client-secret values nor bearer token values appear in the runner argument vector or state. Contractless, mutating, multi-scheme, non-HTTP, unresolved, and cross-platform wires are reported as unsupported rather than inferred healthy. Transport diagnostics and response bodies are not persisted; state contains only the wire, Contract, operation, adapter, security-mechanism identifier, status, outcome, normalized failure code, and UTC timestamp.

The non-HTTP standards are fully covered for definition, selector resolution, compatibility, endpoint selection, materialization, locking, and deterministic planning. They are intentionally not treated as generically live-probeable: AsyncAPI needs broker/protocol delivery semantics and an observable receiver; gRPC needs a request encoded against the selected method and must account for streaming; GraphQL needs a valid operation document, variables, selection set, and inspection of the response `errors` member. Those values are interaction fixtures or application inputs, not Anchor fields. A protocol adapter must add that explicit evidence before claiming live acceptance.

```
# Emit the canonical strict Draft 2020-12 schema
dotx compose schema > composition-v1alpha1.schema.json

# Generate once, commit, and verify without rewriting
dotx compose lock scenario.yaml
dotx compose lock scenario.yaml --check

# Structure, references, selectors, compatibility, and materializer declarations
dotx compose validate scenario.yaml --locked

# Deterministic resolved topology; no runtime mutation
dotx compose plan scenario.yaml --locked
dotx compose plan scenario.yaml --locked --json

# Render runtime actions without state or runtime mutation
dotx compose apply scenario.yaml --locked --dry-run

# Execute, inspect recorded evidence, and clean up owned resources
dotx compose apply scenario.yaml --locked
dotx compose probe scenario.yaml --locked --wire oidc-provider-to-gateway
dotx compose probe scenario.yaml --locked --wire protected-gateway-to-client
dotx compose status scenario.yaml
dotx compose destroy scenario.yaml
```

Apply records each pending instance before runtime mutation, rolls back earlier instances on failure, and preserves recoverable state when rollback is incomplete or `--keep-on-failure` is selected. `apply --resume` requires an identical composition source, resolved plan, runtime options, inputs, ordering, and previously applied runtime state.

Apply, probe, status, and destroy acquire a private per-composition process lock. A concurrent lifecycle command fails immediately rather than racing local state or runtime mutations.

`compose status` reports:

- the recorded phase and native runtime status of each DOT instance;
- recorded wire compatibility, security configuration, selected endpoint, and materializer count; and
- recorded exposed-Anchor bindings; and
- the latest redacted active Contract-probe result for each probed wire.

Wire and exposure entries remain configuration evidence. Only entries under active Contract-probe evidence demonstrate a runtime request and declared OpenAPI response match, bounded by their timestamp and adapter scope.

Fail-closed rules

Apply rejects:

- `UNKNOWN` wire compatibility;
- missing, duplicate, or mismatched IDs and versions;
- ambiguous deployments, bindings, services, ports, or OpenAPI server paths;
- unresolved OpenAPI paths or AsyncAPI channel templates;
- undeclared materializer targets and conflicting target values;
- readiness dependency cycles;
- unsupported secret-store/runtime combinations;
- ungranted OCI capabilities, host namespaces, volumes, or CDI devices;
- repeated DOT instances on deployment mechanisms that cannot isolate their resource names; and
- materialized cross-platform wires without an explicit network binding.

Planning may retain `UNKNOWN` to explain incomplete evidence. Execution never silently upgrades `UNKNOWN` to compatible.

What does not belong in an Anchor

Do not add these as optional Anchor fields:

- hosts, URLs, numeric ports, broker addresses, or Kubernetes names;
- credentials, tokens, certificates, or secret values;
- Camel/Kamelet route syntax;
- transformation logic;
- placement, slice, tenant, trust-zone, latency, or availability policy;
- current health or live authorization state;
- host paths or operator device names; or
- orchestration dependencies.

Use Contracts for interaction semantics, service bindings for deployable ports, composition inputs for operator-controlled configuration, wires for topology, materializers or mediator DOTs for explicit adaptation, and policy engines for deployment/security decisions.

Choosing an extension point

Requirement	Preferred representation
New API/message operation	Add or update the authoritative Contract and select it from an Anchor.
Consumer needs a provider address	Wire materializer targeting declared service configuration.
Payload/protocol transformation	Explicit mediator DOT with two Contracts and two Anchors.

Requirement	Preferred representation
Startup/readiness ordering	Composition `dependsOn`.
Secret or certificate	Reference-only deployment input and platform/external secret provider.
Privileged runtime resource	Typed deployment requirement plus exact composition grant.
Broker service	Explicit wire `endpoint` selecting the broker service port.
Network attachment, eBPF/XDP, WASM import/export, or lifecycle event	Separately reviewed polymorphic binding type when a real use case requires it; do not grow `Anchor` with nullable fields.

6G security composition patterns

The same model supports security-function composition without baking one orchestrator's topology language into every DOT:

- **Zero-trust API chain:** identity-provider and policy-decision DOTs expose token, discovery, and authorization Anchors; an API gateway consumes them and exposes a protected application Anchor. Contracts carry OAuth/OpenID Connect and application semantics, while the composition supplies client-specific secret references and issuer URLs.
- **Closed-loop detection and response:** telemetry, inference, CACAO interpretation, and OpenC2 actuation remain separate DOTs. HTTP and MQTT Contracts define the boundaries; the wire selects the broker endpoint and supplies only deployment-specific channel values.
- **Multi-domain federation:** peer DOT instances can have reciprocal wires while `dependsOn` contains only real readiness dependencies. Trust-domain, roaming, and federation policy belongs in policy inputs or a dedicated policy DOT, not in the Anchor.
- **Near-real-time RAN control:** E2 or management APIs are Contract-backed Anchors. Host networking, accelerators, eBPF/XDP attachments, and radio-device access are explicit deployment requirements and grants, never inferred from a wire.
- **Observability and digital twins:** metrics, logs, events, and integrity APIs are distinct Anchors. A storage or message broker is selected explicitly as a connection endpoint; a shared host path is not treated as a portable integration boundary.

For cross-domain 6G deployments, a successful plan proves model consistency, not trust. Admission policy must still evaluate publisher identity, artifact provenance, attestation, slice/tenant placement, data-sovereignty constraints, and least-privilege grants before execution. Runtime acceptance should then probe the selected Contract operation over the actual network path.

Troubleshooting

Symptom	What to check
Anchor UUID is not found	The wire references the pinned DOT version, the UUID is stable, and the Anchor is present in that version's `dot.json`.

Symptom	What to check
Contract or selector cannot be resolved	The composition maps the exact Contract UUID to a readable source, the source document's `id` matches it, and the selector uses the standard-specific syntax.
Anchor has no deployable endpoint	The selected deployment contains exactly one service binding for the Anchor, and its named port exists on that same service.
Compatibility is `UNKNOWN`	Inspect `compatibilityReasons` in `compose plan --json`; add authoritative Contract evidence or use an explicit mediator. Do not override the result.
Materializer target is rejected	The target service declares the environment variable or config-file path, the JSON Pointer is valid, and no other wire writes a conflicting value.
Security remains unresolved	Bind a Contract-required supported scheme, declare all participant variables, and supply sensitive values by `secretRef` rather than plaintext.
Apply reports the composition is busy	Another apply, status, or destroy holds the per-composition lock. Let that command finish; do not delete the lock while a lifecycle process is running.
Status shows a configured wire but no active evidence	Run `dotx compose probe scenario.yaml --wire <wire>`. If the wire is skipped, its operation requires a dedicated adapter or application-level acceptance test. Configuration evidence alone is not reachability evidence.
An OAuth probe reports `AUTHENTICATION_ERROR`	Check the bound token URL and client ID, the referenced Secret name/key or environment-store source, required scopes, issuer reachability, and client authorization. Dotx deliberately does not persist the token endpoint's body.

Executable examples

- `DOT-Testing/conformance/anchors/` covers all four Contract standards, contractless Anchors, validation failures, bindings, and compatibility.
- `DOT-Testing/e2e/api-gateway-oidc/` is the canonical authenticated request/response lifecycle example.
- `DOT-Testing/composition/poc6-closed-loop/` demonstrates an explicit MQTT broker endpoint and OpenC2/AsyncAPI operations.
- `DOT-Testing/composition/federation-peers/` demonstrates reciprocal wires without false readiness cycles.
- `DOT-Testing/e2e/poc1-ric-aaa/` demonstrates identity, OPA authorization, forwarding, metrics feedback, and deny-list update across seven wires.
- `DOT-Testing/interoperability/pocs.yaml` is the evidence ledger for all eleven MARE PoCs and records every remaining acceptance gap.

Run the repository gate with:

```
task testing:conformance
```

The source-adapted E2E tests prove only their documented boundaries. They do not claim partner-runtime, live-RIC, production-network, trust-policy, or SFC acceptance unless that exact behavior is asserted by the scenario.

Verification and examples

DOT-Design keeps model correctness, resolved planning, deployment dry-runs, and live behavioral acceptance as separate evidence levels. A passing lower level never implies a higher one.

Evidence model

Level	Demonstrates	Does not demonstrate
Model and schema conformance	`dot.json` shape, Bean Validation constraints, stable references, selectors, service-port bindings, and generated-model compatibility	Contract payload resolution or multi-DOT topology
Contract resolution and compatibility	Selected operations exist and normalized semantic, interaction, protocol, media, schema, and security facts can be evaluated conservatively	Runtime reachability or valid credentials
Composition validation and plan	Pinned DOTs, wires, endpoints, materializers, parameters, grants, and readiness ordering resolve deterministically	Successful deployment
Lifecycle dry-run	The reference executor accepts the supported profile and renders runtime actions without writing composition state	Workload startup or traffic
Source-adapted E2E	Pinned upstream logic or artifacts execute behind explicit test adapters and satisfy the assertions documented by that scenario	Unmodified partner runtime, production infrastructure, or full PoC acceptance
Live production acceptance	The intended workloads, network, identity, policy, and Contract operations work in the target environment	Nothing beyond the probes and acceptance criteria actually executed

Primary verification commands

Command	Scope	External requirements
`mvn -pl DOT-Domain test`	Domain value objects, manifests, Anchors, and aggregate validation	Java and Maven only
`go -C DOT-CLI test ./…`	CLI parsing, validation, planning, deployment adapters, and lifecycle behavior	Go; individual tests use doubles where appropriate

Command	Scope	External requirements
`task testing:dots:repository`	Every definition admitted by `DOT-Repository/dot-repository.yaml`: catalogue identity, scope/path isolation, schema validity, CLI cross-references, and uniqueness	Go and the checked-out `DOT-CLI`, `DOT-Repository`, and `DOT-Schema` revisions
`task testing:compose:contract-standards`	Two real DOT definitions connected through OpenAPI YAML/JSON, AsyncAPI YAML/JSON, Protobuf/gRPC, and GraphQL SDL; asserts selectors, participant interactions, endpoints, and materializers	`dotx`, `jq`, and the checked-out repository/schema revisions
`task testing:compose:runtime-configuration`	The same catalogued DOT receives declared environment and configuration-file inputs through Docker image, Docker Compose, Kubernetes manifest, Helm chart, and Vagrantfile dry-run adapters	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:compose:service-workloads`	A two-service DOT receives isolated defaults, overrides, configuration files, and secret references through exact Compose, Kubernetes, and Helm workload bindings	`dotx`, `jq`, and runtime CLIs in dry-run rendering mode
`task testing:cli:coverage`	Complete DOT-CLI test suite plus a machine-readable statement-coverage profile	Go; writes reports below `DOT-Testing/e2e/target/`
`task testing:conformance`	Cross-module Anchor and composition gate, including all deterministic PoC fixtures and lifecycle dry-runs	Java, Maven, Go, Task, and fixture-specific local tools
`mvn clean install`	Full Maven reactor, Registry integration tests, schema generation, and SBOMs	MongoDB for Registry tests
`task testing:e2e:<scenario>`	Scenario-specific live behavior	Read that scenario's README first; commonly Docker and network access

Test asset layout

```
DOT-Testing/
 ├──── acceptance/        versioned consumer profiles and golden API responses
 ├──── conformance/       deterministic valid and invalid DOT fixtures
 ├──── composition/       read-only validate, plan, and dry-run scenarios
 ├──── e2e/               opt-in deployable behavioral scenarios
 └──── interoperability/  revision-pinned evidence and remaining-gap ledger
```

`testing` is repository-level infrastructure, not a Maven module and not a deployable DOT. Authoritative test-only definitions live under `DOT-Repository/DOTs/testing/` and are selected from compositions with `repository://<dot-id>@<version>`. Intentionally invalid conformance inputs stay under `DOT-Testing/conformance/`; they must never be admitted to the repository catalogue. Generated secrets, logs, reports, coverage profiles, and materialized configuration belong under scenario-local `target/` directories and must not be committed.

The focused `task testing:anchors:acceptance` gate verifies the published DOT Anchor consumer profile and its complete cross-standard golden Registry responses without MongoDB or a deployed workload.

The focused `task testing:compose:contract-standards` gate complements that API profile with an executable composition matrix. It proves all six supported standard/format representations between the same provider and consumer DOTs; it does not promote planning evidence to live protocol or application acceptance.

The runtime-configuration gate separately proves that selecting another executable deployment does not discard a service's declared environment or configuration-file surface. Runtime-native storage, secret, privilege, and device semantics retain dedicated tests rather than being flattened into a misleading universal volume abstraction.

Coverage claims and gates

Coverage is reported by evidence dimension; the project does not collapse different meanings of "covered" into one percentage:

Dimension	Gate	Interpretation
Repository inventory	`dotx repository test`, `dotx repository audit`, and the DOT repository conformance workflow	Must be 100%: every catalogue entry is discovered, parsed, schema-valid, cross-reference-valid, uniquely identified, and stored in its declared scope; no deployable copy may remain outside `DOT-Repository`.
CLI statements	`task testing:cli:coverage`	Measured and published as an artifact. The central repository-command package has an 80% regression floor; the complete CLI percentage remains visible. This is a code-execution metric, not proof that a workload or security control works.
Composition scenarios	`task testing:e2e:all:dry-run`	Every listed deterministic scenario must validate, plan, and complete its supported lifecycle dry-run.

Dimension	Gate	Interpretation
PoC interoperability	DOT-Testing/interoperability/pocs.yaml plus scenario-specific live workflows	Each of the eleven PoCs has traceable evidence, while remaining adapters, unavailable partner components, and unproved production behavior stay explicit.

The repository test gate fails on partial inventory, duplicate identities, mis-scoped definitions, or invalid DOTs. The workspace audit separately classifies uncatalogued, identical, divergent, and malformed definitions and fails on every deployable `dot.json` outside `DOT-Repository`. `DOT-Testing/conformance/` fixtures are reported but exempt because invalid definitions are their purpose. During the staged migration, run:

```
dotx repository audit --repository DOT-Repository --workspace . --json
```

The statement report deliberately has no artificial 100% threshold: a high line percentage cannot replace negative tests, lifecycle checks, live probes, or partner acceptance.

Canonical Anchor scenario

`DOT-Testing/e2e/api-gateway-oidc/` is the primary tutorial and lifecycle fixture. Its composition resolves three independently valid, test-scoped definitions from `DOT-Repository`:

- Keycloak provides OpenID Connect discovery and token operations;
- APISIX requires discovery information and provides a protected operation; and
- an OAuth client requires the token and protected operations.

The composition uses three OpenAPI Contracts, three wires, explicit security configuration, Kubernetes Secret references, configuration-file and environment materializers, readiness dependencies, and one exposed Anchor.

```
task testing:compose:api-gateway-oidc
task testing:e2e:anchors:api-gateway-oidc:dry-run

# Opt-in live execution
task testing:e2e:anchors:api-gateway-oidc
```

Read DOT Anchors and Composition alongside `DOT-Testing/e2e/api-gateway-oidc/scenario.yaml`.

MARE PoC-derived coverage

The interoperability ledger pins the source revision used for every analysis, records the modeled interactions, identifies the executable fixtures, and keeps remaining acceptance gaps explicit. The current repository contains source-adapted evidence for all eleven PoCs; this is not a claim of unmodified partner-runtime acceptance.

PoC	Composition focus	Fixture
1	OIDC identity, OPA authorization, RIC forwarding, metrics feedback, and deny-list update	`DOT-Testing/composition/poc1-ric-aaa/`
2	Repeated Flower trainers over gRPC/Protobuf and exposed BentoML inference	`DOT-Testing/composition/poc2-federated-inference/`
3	Five-stage contractless JSON insider-threat analytics dataflow	`DOT-Testing/composition/poc3-insider-threat-pipeline/`
4	Prometheus metrics, Elasticsearch logs, Grafana, and Kibana configuration	`DOT-Testing/composition/poc4-continuous-monitoring/`
5	Monitoring-to-analysis-to-assessment-to-mitigation security pipeline	`DOT-Testing/composition/poc5-security-pipeline/`
6	OpenAPI plus brokered AsyncAPI/MQTT, CACAO, OpenC2, and NWDAF remediation	`DOT-Testing/composition/poc6-closed-loop/`
7	Least-privilege packet capture, explicit CloudEvents mediation, and trust scoring	`DOT-Testing/composition/poc7-trust-scoring/`
8	cAdvisor/Prometheus observability and physical/twin integrity checks	`DOT-Testing/composition/poc8-ndt-observability/` and `poc8-twin-integrity/`
9	Amarisoft/Open5GS collection, InfluxDB storage, and data export	`DOT-Testing/composition/poc9-edge-ddos-collection/`
10	OIDC-protected capability exposure, SPARQL, OPA policy decisions, and observability	`DOT-Testing/composition/poc10-secure-capability-exposure/`
11	Reciprocal federation peers and local-core adapters with separate readiness edges	`DOT-Testing/composition/federation-peers/`

The authoritative coverage and gap list is `DOT-Testing/interoperability/pocs.yaml`. The official project descriptions are available from the MARE PoC catalogue.

Negative evidence is intentional

Rejected fixtures are part of the design evidence. They demonstrate that the model does not invent connectivity where the pinned source lacks a declared consumer input, does not treat stdout or a host path as a portable connection, and does not silently grant privileged runtime access.

Examples include:

- PoC7 rejects a collector-to-scorer wire when the collector declares no such Anchor or configuration input;
- PoC8 rejects an undeclared Prometheus-to-NDT handoff; and

- invalid conformance fixtures cover missing Contracts, selectors, ports, duplicate bindings, and incompatible Anchor pairs.

Failing closed is a successful test outcome when the available evidence cannot support the proposed composition.

Adding or promoting a scenario

1. Pin the exact external source revision used as evidence.
2. Document what the source actually produces and consumes.
3. Add reusable deployable definitions to `DOT-Repository/DOTs/testing/`, record their evidence and purpose in `dot-repository.yaml`, and select them by exact `repository://<uuid>@<version>` identity.
4. Define Contracts only for supported, observable interactions.
5. Use contractless Anchors only when no supported Contract standard applies.
6. Declare all service ports, configuration targets, privileges, and devices.
7. Add negative fixtures for tempting but unsupported connections.
8. Start under `DOT-Testing/composition/` with validate and plan assertions.
9. Add a lifecycle dry-run before any live execution.
10. Promote to `DOT-Testing/e2e/` only with isolated resources, explicit prerequisites, reliable cleanup, and externally observable assertions.
11. Record what remains unproved in the interoperability ledger.

Interpreting results



A successful plan proves model consistency. A successful apply proves that the supported deployment adapters completed. Only an active scenario probe can prove that traffic, authentication, authorization, policy, or security behavior worked as intended.

DOT-Domain

Domain model

`DOT-Domain` is the Java library that defines the portable DOT aggregate and its validation rules. It contains no Spring Boot application or web layer.

Aggregate overview

Type	Responsibility
`Dot`	Aggregate root: identity, version, description, classification groups, metadata, Skills, Contracts, Anchors, and Runtimes.
`Skill`	Standalone human-readable capability description.
`Contract`	Standalone authoritative OpenAPI, AsyncAPI, Protobuf, or GraphQL document.
`Anchor`	Embedded stable logical point through which the DOT participates in composition.
`Runtime`	A target `Platform` plus one or more compatible deployment definitions.
`Deployment`	Stable deployment ID, one polymorphic manifest, and the logical services it materializes.
`DeploymentService`	Service identity, host, named ports, Contract references, Anchor bindings, and its supported configuration surface.

`Dot`, `Skill`, and `Contract` are MongoDB root documents. Other types are embedded value objects owned by a DOT.

Identity, versioning, and references

Root document IDs are UUIDs. Equality for `Dot`, `Skill`, and `Contract` is ID-based so changes to mutable metadata or collections do not destabilize sets or persistence identity.

`Version` accepts Semantic Versioning 2.0.0 values and the special `latest` selector. It serializes as a string and implements SemVer precedence; `latest` sorts after concrete versions. A new `Dot` or `Contract` defaults to `0.0.0`, while `Platform.version` defaults to `latest` so no platform-specific release is implied.

`Dot.skills`, `Dot.contracts`, and `DeploymentService.contracts` use Spring Data MongoDB document references. JSON serializes those references as ID-only objects. Referenced documents must be saved independently; persistence does not cascade automatically.

Uniqueness rules are:

- `Skill.name` is globally unique;

- `Dot` is unique by `(name, version)`; and
- `Contract` is unique by `(name, version)`.

Contracts

Standard	Payload format	Anchor selector
`OPENAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`ASYNCAPI`	`YAML` or `JSON`	RFC 6901 URI-fragment JSON Pointer to an Operation Object
`PROTOBUF`	`PROTOBUF`	Fully qualified service method
`GRAPHQL`	`GRAPHQL`	Root-field schema coordinate

`Contract.isFormatCompatibleWithStandard()` rejects mismatched standard/format pairs. Payload parsing and selected-element existence are Registry concerns; the domain model validates selector syntax and references within the aggregate.

Anchors and bindings

A contract-backed Anchor contains:

- a stable UUID and display name;
- a `PROVIDES` or `REQUIRES` role;
- optional semantic-type URIs; and
- a Contract UUID plus standard-specific selector.

It omits direction, protocol, and media type because those facts come from the selected Contract operation. A contractless Anchor must declare `direction`, at least one protocol, and at least one media type explicitly.

`DeploymentService.anchorBindings` maps an Anchor UUID to a named `ServicePort` on that same service. An Anchor contains no host, numeric port, credential, placement, or live policy state.

The class-level `@ValidAnchor` constraint delegates cross-reference validation to `AnchorValidator`. It rejects:

- duplicate Anchor UUIDs within one DOT;
- Contract references not declared by the same DOT;
- selectors invalid for the referenced Contract standard;
- bindings to undeclared Anchors;
- bindings to port names absent from that service; and
- duplicate bindings of one Anchor within the same service.

See DOT Anchors and Composition for authoring and wiring examples.

Runtime two-layer model

`Runtime` separates infrastructure from deployment mechanism:

Enum	Used by	Answers
`PlatformEngine`	`Platform.engine`	Where the workload runs: `DOCKER`, `KUBERNETES`, `VAGRANT`, or `WEBASSEMBLY`.
`DeploymentManifestType`	`DeploymentManifest.type`	How it is deployed: Docker image, Docker Compose, Kubernetes manifest, Helm chart, Vagrantfile, or WASM module.

Every `DeploymentManifestType` declares its required `PlatformEngine`. `Runtime.isDeploymentPlatformAligned()` validates every deployment against the runtime platform. `Runtime.isDeploymentIdsUnique()` enforces stable deployment selection within that runtime.

`Deployment.services` is a set. `DeploymentService` uses ID-based equality, so logical service IDs remain unique within the deployment.

Deployment manifests

`DeploymentManifest` is polymorphic JSON dispatched by its existing `type` property.

Type	Java subtype	Important fields
`DOCKER_IMAGE`	`DockerImage`	Image reference and ordered dotenv payloads
`DOCKER_COMPOSE`	`DockerCompose`	Complete YAML/JSON payload and ordered dotenv payloads
`KUBERNETES_MANIFEST`	`KubernetesManifest`	Complete YAML/JSON manifest payload
`HELM_CHART`	`HelmChart`	Repository, chart or OCI reference, version, and ordered values payloads
`VAGRANTFILE`	`Vagrantfile`	Complete Ruby DSL payload
`WASM_MODULE`	`WebAssemblyModule`	Module URI

Docker Compose and Kubernetes manifest payloads accept YAML or JSON only. Helm values also preserve YAML/JSON order: later entries override earlier entries as with repeated Helm `-f` arguments.

Runtime instance concerns do not belong in a manifest. For example, a Helm release name and namespace are selected by deployment tooling, not embedded in `HelmChart`.

Service configuration declarations

Every `DeploymentService` owns one `DeploymentConfig` containing:

- environment variables, including default, required, and sensitive metadata;
- configuration-file paths, content type, defaults, and sensitivity;

- logical volume mounts;
- logical CNCF CDI device requirements; and
- optional OCI capability and host-namespace requirements.

These are supported inputs and requirements, not applied runtime state. Operator values, host paths, concrete CDI device names, and permission grants belong to deployment or composition input.

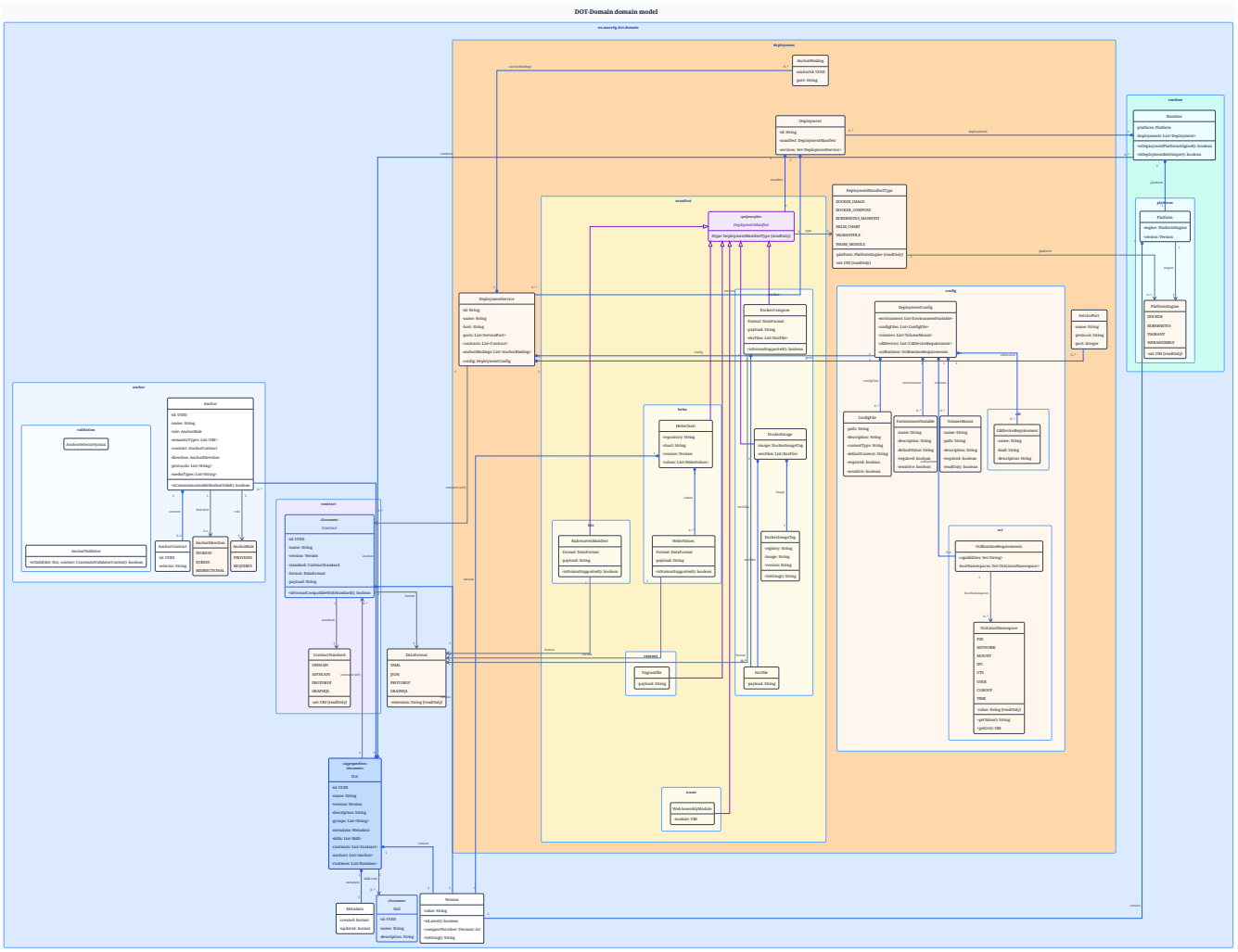
OCI requirements use standard `CAP_*` capability names and typed Linux namespaces. The model deliberately has no unrestricted `privileged` flag.

A CDI requirement declares a stable logical name and a `vendor.example/class` kind. A composition binds it to one exact `vendor.example/class=device` identifier. CDI installation, allocation, and authorization remain operator responsibilities.

Serialization and persistence

- Collections have initialized builder defaults and do not become `null` when constructed through Lombok builders.
- `Version` is JSON-serialized and MongoDB-persisted as a normalized string.
- Both Registry application configurations use standard UUID representation.
- Referenced `Skill` and `Contract` objects expose only `id` when nested in DOT JSON.
- Embedded values do not carry MongoDB `@Document` annotations.

Class diagram



The diagram is generated from current domain sources during the DOT-Domain build. Open it separately for the full-resolution view.

Build and test

```
# Unit tests only; no Spring context or MongoDB
mvn -pl DOT-Domain test

# Regenerate schema and diagram through the reactor lifecycle
mvn -pl DOT-Domain clean install
```

Representative tests cover version precedence, reference serialization, manifest polymorphism, runtime/manifest alignment, deployment configuration, Anchor rules, and entity equality.

Related pages

- Architecture
- DOT Anchors and Composition
- DOT-Registry

DOT-Registry

Registry service

`DOT-Registry` is the Spring Boot service around the DOT domain model. Its current public surface is deliberately read-only: consumers can discover Anchors, retrieve Contracts, and resolve selected Contract elements without copying derived facts into the stored DOT.

Responsibilities

- Persist `Dot`, `Skill`, and `Contract` root documents in MongoDB.
- Resolve embedded Anchor references within a stored DOT.
- Load the referenced Contract document.
- Parse the selected OpenAPI, AsyncAPI, Protobuf, or GraphQL element.
- Return a stable normalized response for composer and tooling consumers.
- Return stable machine-readable errors for missing or invalid data.

The Registry does not allocate runtime endpoints, resolve credentials, deploy workloads, or persist composition wires.

Run locally

Start MongoDB on `localhost:27017`, then run:

```
mvn -pl DOT-Registry spring-boot:run
```

The application uses database `dots`. Automatic index creation is enabled and UUIDs use the MongoDB standard representation.

Spring Boot Actuator is included. Availability and exposure of individual Actuator routes follow the active Spring Boot configuration.

Read API

Route	Result
`GET /dots/{dotId}/anchors`	All logical Anchors embedded in the DOT.
`GET /dots/{dotId}/anchors/{anchorId}`	One logical Anchor.
`GET /dots/{dotId}/anchors/resolved`	All Anchors with normalized Contract metadata and selected elements.
`GET /dots/{dotId}/anchors/{anchorId}/resolved`	One resolved Anchor.
`GET /contracts/{contractId}`	The complete authoritative Contract, including its payload.

Prefer a resolved Anchor route when making composition decisions. Use the Contract route only when the complete authoritative payload is required.

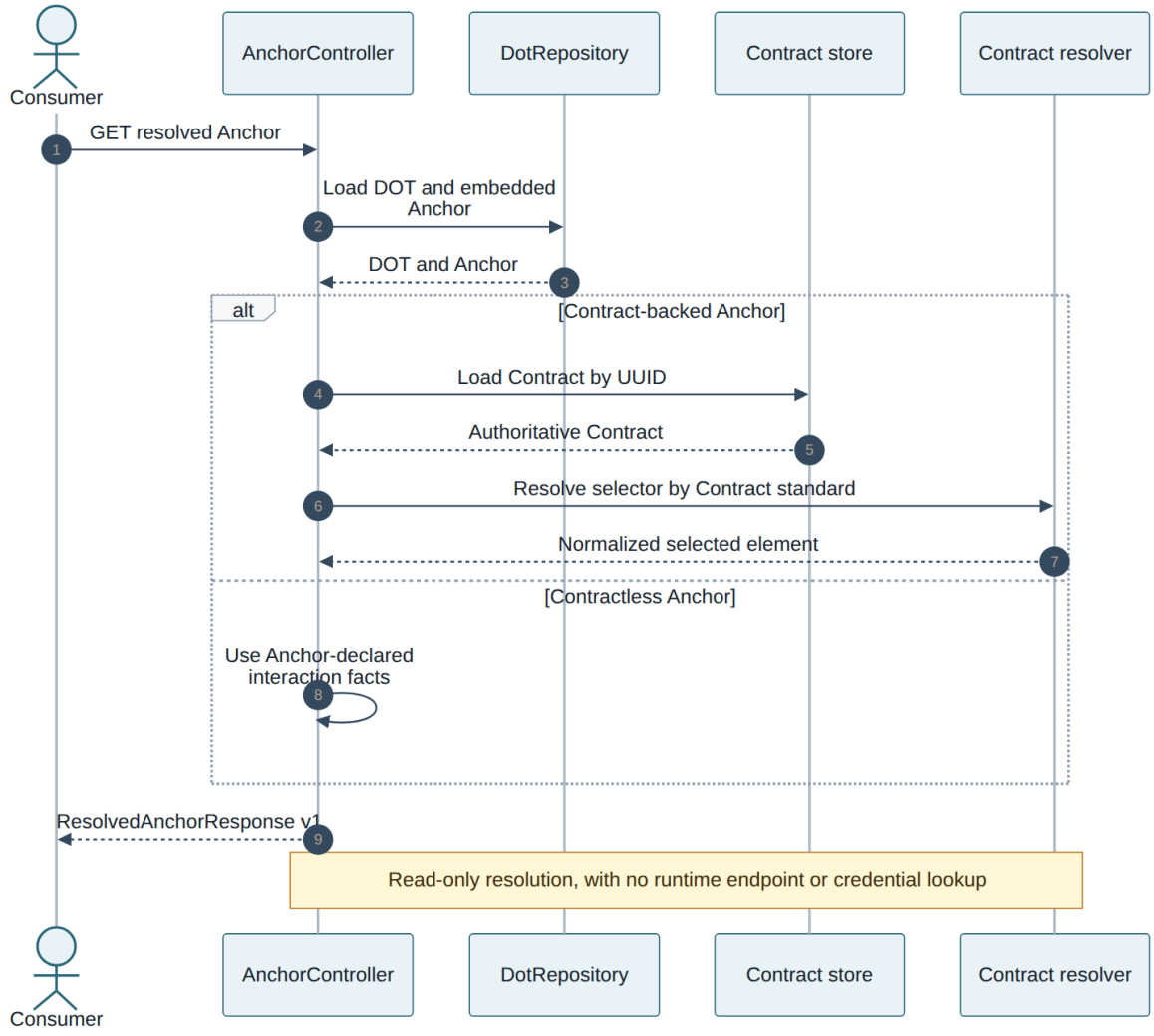


Figure 7. Resolved Anchor request

Resolved Anchor response

```
{
  "apiVersion": "1",
  "anchor": {
    "id": "a0000000-0000-4000-8000-000000000001",
    "name": "submit-alert-http",
    "role": "PROVIDES",
    "semanticTypes": ["urn:mare:security:alert"],
    "contract": {
      "id": "11111111-1111-4111-8111-111111111111",
      "selector": "#/paths/~1alerts/post"
    }
  },
  "contract": {
    "id": "11111111-1111-4111-8111-111111111111",
    "name": "alert-http-api",
    "version": "1.0.0",
```

```
    "standard": "OPENAPI",
    "format": "YAML"
  },
  "selectedElement": {
    "selector": "#/paths/~1alerts/post",
    "identifier": "submitAlert",
    "interaction": "REQUEST_RESPONSE",
    "protocols": ["HTTPS"],
    "mediaTypes": ["application/json"],
    "inputSchemas": ["#/components/schemas/Alert"],
    "outputSchemas": ["#/components/schemas/Receipt"],
    "securitySchemes": ["oauth2"]
  }
}
```

`apiVersion` versions the normalized response independently of the DOT version. The nested Contract intentionally excludes its potentially large payload. For a contractless Anchor, `contract` and `selectedElement` are `null`. A dangling Contract reference is an error and is never presented as a contractless Anchor.

Contract resolution

Standard	Selector	Normalized facts
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`REQUEST_RESPONSE`, server protocols, request/response media types and schema references, and security scheme names
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object	`SEND` or `RECEIVE`, server/binding protocols, message media type and schema references, and security scheme names
Protobuf	Fully qualified service method	Unary, client-streaming, server-streaming, or bidirectional-streaming interaction; gRPC protocol and request/response message types
GraphQL	Root-field schema coordinate	`QUERY`, `MUTATION`, or `SUBSCRIPTION`; argument and result type expressions

OpenAPI and AsyncAPI payloads may be JSON or YAML. Protobuf payloads are `.proto` source, and GraphQL payloads are SDL. Local `$ref` values used by the supported resolvers are normalized from the authoritative document; unsupported or invalid constructs fail with an explicit error.

Error contract

Errors use a stable two-field JSON body:

```
{
  "code": "SELECTOR_NOT_FOUND",
  "message": "selector does not identify an element in the contract:
```

```
#/paths/~1missing/get"
}
```

Codes	HTTP status
`DOT_NOT_FOUND`, `ANCHOR_NOT_FOUND`, `CONTRACT_NOT_FOUND`	`404 Not Found`
`INVALID_CONTRACT`, `INVALID_SELECTOR`, `SELECTOR_NOT_FOUND`, `UNSUPPORTED_CONTRACT_FEATURE`	`422 Unprocessable Content`

Consumers should branch on `code`; `message` is diagnostic text.

Compatibility evaluation

The Registry contains a conservative three-state evaluator for resolved Anchor pairs:

- `COMPATIBLE` — every currently evaluated rule is satisfied;
- `INCOMPATIBLE` — at least one deterministic mismatch exists; and
- `UNKNOWN` — no mismatch was proved, but required evidence or an external evaluator is missing.

It evaluates role, semantic type, interaction, protocol, media type, schema identity, and security requirements. Exact schema identifiers prove compatibility only when both Anchors use the same authoritative Contract UUID. Cross-Contract structural comparison, ontology reasoning, transformations, and deployment-environment security satisfaction remain `UNKNOWN`.

The evaluator is currently an internal service; there is no public compatibility route. This avoids publishing a request identity and policy contract before those boundaries are stable.

Runtime handoff

Registry resolution stops at logical facts. `AnchorBinding` maps an Anchor UUID to a `DeploymentService` and named `ServicePort`; deployment tooling then maps that service/port to a live endpoint.

Hosts, allocated ports, broker addresses, credentials, certificates, network slices, and current policy state remain deployment-instance data and are never copied into a logical Anchor.

Persistence

Context	MongoDB database
Application	`dots`
Integration tests	`dots-test`

Both configurations set `spring.mongodb.representation.uuid: standard`. Automatic index creation enforces the unique `Skill.name` index and unique `(name, version)` indexes for DOTs and Contracts.

Spring Data document references do not cascade. Tests and clients must persist referenced Skills and Contracts before saving a DOT that references them.

Build and test

Registry tests use a real MongoDB instance; no embedded database is configured.

```
cd DOT-Registry
./mvnw test
```

Repository integration tests clear their collections before and after each test class. Resolver and controller tests cover supported standards, malformed payloads, missing selectors, normalized responses, and error mappings.

`DOT-Registry/pom.xml` intentionally overrides annotation processors to include both Lombok and `spring-boot-configuration-processor`; both are required.

Operational boundary

The current service is suitable as a reference read API and persistence layer. Production deployment still requires environment-specific authentication and authorization, parser/input limits, audit and metrics policy, availability, backup/restore, and API compatibility governance.

Related pages

- Architecture
- DOT Anchors and Composition
- DOT-Domain model
- SFC integration handoff

Anchor consumer acceptance

The DOT Anchor consumer profile turns the Registry integration boundary into a repeatable acceptance contract. It is intended for composition engines, catalogues, deployment controllers, and the Security Function Composer team. It does not prescribe their internal wire or orchestration model.

The current profile is **DOT Anchors consumer profile v1**. Its status is release-candidate and its Registry response version is `apiVersion: "1"`.

Profile contents

Artifact	Purpose
`DOT-Testing/acceptance/dot-anchors/v1/profile.yaml`	Machine-readable manifest that binds the profile version to its source fixture, golden responses, error families, and verification gates.
DOT Registry Anchor Read API v1	OpenAPI 3.1 contract for raw and resolved Anchor reads, schemas, status codes, and stable error codes.

Artifact	Purpose
`DOT-Testing/acceptance/dot-anchors/v1/expected/`	Complete resolved-Anchor responses generated from the OpenAPI, AsyncAPI, Protobuf, GraphQL, and contractless conformance Anchors.
`DOT-Testing/acceptance/dot-anchors/v1/consumer-checklist.md`	Manual checklist for discovery, resolution, composition-boundary checks, and joint runtime acceptance.
`AnchorConsumerAcceptanceTests`	Executable drift guard connecting the profile to the Java DTOs, Contract resolvers, error taxonomy, and OpenAPI document.

The profile reuses `DOT-Testing/conformance/anchors/valid/`; it does not copy the authoritative Contract payloads into a second fixture. The golden responses are consumer-facing outputs, not alternative Contract definitions.

What a conforming consumer must do

A conforming v1 consumer must:

- address DOTs and Anchors by UUID and reject missing references;
- recognize resolved response version `1`;
- distinguish contract-backed and contractless Anchors;
- use the selected Contract element instead of expecting duplicated protocol, schema, media-type, or security metadata in a contract-backed Anchor;
- support the stable `404` and `422` error families;
- fail closed on selectors or Contract features it cannot resolve;
- keep runtime endpoints, credentials, wires, parameter values, readiness, and policy outside the DOT Anchor; and
- distinguish `UNKNOWN` compatibility from `COMPATIBLE`.

The complete operational checklist is maintained with the executable profile so that it can be attached to an integration report without copying this page.

Run the acceptance gate

From the repository root:

```
task testing:anchors:acceptance
```

This focused gate requires Java and Maven but no MongoDB or deployed security function. It proves that every golden response can still be produced from the authoritative conformance fixture and that the published OpenAPI operations are resolvable by the same Contract resolver used by the Registry.

For the full deterministic cross-module gate, including DOT JSON Schema, CLI, composition

planning, and lifecycle dry-runs:

```
task testing:conformance
```

For behavioral evidence, continue with the APISIX, Keycloak, and OAuth-client scenario documented in Verification and examples.

Versioning policy

The profile version and `ResolvedAnchor.apiVersion` are explicit compatibility boundaries:

- compatible clarifications, additional fixtures, and new optional examples may extend `v1` without changing its response version;
- removing or renaming a response field, changing its meaning or nullability, changing stable error semantics, or narrowing an accepted value requires a new response and acceptance-profile version;
- adding a Contract standard or a new normalized interaction requires a v1 golden case before it can be claimed as supported;
- old profiles remain executable during a migration window; and
- individual DOT semantic versions do not replace the consumer profile version.

Anchor UUID continuity is evaluated independently: the same logical connection capability keeps its UUID across compatible DOT revisions. An incompatible replacement receives a new UUID and must not be hidden by a profile-version change.

Joint SFC acceptance

DOT-Design owns and executes this profile. The Security Function Composer remains unchanged until its maintainers choose to integrate it. A joint session should record:

1. the exact DOT-Design and SFC revisions;
2. the profile version and Registry API version;
3. the completed consumer checklist;
4. the DOT and Anchor UUIDs used by each wire;
5. the Contract operation selected for each side;
6. planning, deployment, and active runtime-probe results separately; and
7. every unsupported requirement or adapter boundary.

Passing the deterministic profile is necessary but does not prove endpoint reachability, credential validity, authorization policy, or live security behavior. Those claims require explicit runtime probes and remain separate from the Anchor model.

SFC integration handoff

This page is the implementation handoff from DOT-Design to Security Function Composer (SFC) maintainers. It defines the DOT-owned discovery and resolution contract, identifies the information that remains composer-owned, and records the evidence available in this repository.



This handoff is informational. It does not prescribe or modify the SFC-owned schema. SFC continues to own *wires*, *exposed anchors*, transformations, and composition policy. DOT-Design owns the `anchors` model, Contract resolution, deployment bindings, and their validation rules.

For the complete ownership model, DOT JSON examples, wire and materializer semantics, lifecycle behavior, and extension guidance, see DOT Anchors and Composition. The versioned, executable integration contract is documented in DOT Anchor consumer acceptance.

Integration boundary

A DOT declares stable logical `anchors`. A contract-backed anchor references an authoritative contract already declared by that DOT and selects one operation. It does not repeat direction, transport, media type, message schema, or security facts from the contract.

```
{
  "id": "a0000000-0000-4000-8000-000000000001",
  "name": "submit-alert-http",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:alert"],
  "contract": {
    "id": "11111111-1111-4111-8111-111111111111",
    "selector": "#/paths/~1alerts/post"
  }
}
```

A contractless anchor is reserved for an interaction without a supported authoritative contract. Only that form declares the missing communication facts explicitly:

```
{
  "id": "a0000000-0000-4000-8000-000000000006",
  "name": "packet-stream",
  "role": "PROVIDES",
  "semanticTypes": ["urn:mare:security:packet-stream"],
  "direction": "EGRESS",
  "protocols": ["TCP"],
  "mediaTypes": ["application/octet-stream"]
}
```

`anchors` is the normative DOT field. SFC may expose an anchor or connect two anchors, but DOT does not define SFC's wire/exposure object shape.

Stable references

SFC should persist the DOT identity/version selected for a composition and the anchor UUID. Anchor names are display labels and may change.

The following is a non-normative illustration, not an SFC schema proposal:

```
from:
  dotId: d0000000-0000-4000-8000-000000000001
  dotVersion: 1.0.0
  anchorId: a0000000-0000-4000-8000-000000000002
to:
  dotId: d0000000-0000-4000-8000-000000000001
  dotVersion: 1.0.0
  anchorId: a0000000-0000-4000-8000-000000000003
```

The same logical anchor keeps its UUID across compatible DOT versions. Renames and documentation changes do not create a new identity. An incompatible semantic replacement uses a new UUID. Cross-version enforcement and future deprecation/replacement metadata remain publishing-governance work.

Selector conventions

Contract standard	Anchor selector
OpenAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object, for example `#/paths/~1alerts/post`
AsyncAPI	RFC 6901 URI-fragment JSON Pointer to an Operation Object, for example `#/operations/publishAlert`
Protobuf	Fully qualified service method, for example `mare.security.AlertService.PublishAlert`
GraphQL	Root-field schema coordinate, for example `Mutation.publishAlert`

Selector syntax is checked in the DOT model. Parsing the authoritative payload and proving that the selected element exists occurs in the Registry.

URI-fragment percent encoding is required for characters which cannot appear literally. For example, `/routes/{id}` is represented by `#/paths/~1routes~1%7Bid%7D/put`; the Registry decodes the URI fragment before applying the JSON Pointer.

Registry API

The consumer endpoints are:

- `GET /dots/{dotId}/anchors`
- `GET /dots/{dotId}/anchors/{anchorId}`
- `GET /dots/{dotId}/anchors/resolved`

- `GET /dots/{dotId}/anchors/{anchorId}/resolved`
- `GET /contracts/{contractId}`

Prefer the resolved endpoints for composition decisions. Their versioned response contains contract metadata but intentionally omits the potentially large payload:


```
{
  "apiVersion": "1",
  "anchor": {
    "id": "a0000000-0000-4000-8000-000000000001",
    "name": "submit-alert-http",
    "role": "PROVIDES",
    "semanticTypes": ["urn:mare:security:alert"],
    "contract": {
      "id": "11111111-1111-4111-8111-111111111111",
      "selector": "#/paths/~1alerts/post"
    }
  },
  "contract": {
    "id": "11111111-1111-4111-8111-111111111111",
    "name": "alert-http-api",
    "version": "1.0.0",
    "standard": "OPENAPI",
    "format": "YAML"
  },
  "selectedElement": {
    "selector": "#/paths/~1alerts/post",
    "identifier": "submitAlert",
    "interaction": "REQUEST_RESPONSE",
    "protocols": ["HTTPS"],
    "mediaTypes": ["application/json"],
    "inputSchemas": ["#/components/schemas/Alert"],
    "outputSchemas": ["#/components/schemas/Receipt"],
    "securitySchemes": ["oauth2"]
  }
}
```


For a contractless anchor, `contract` and `selectedElement` are `null`. A dangling contract reference is an error and is never represented as a contractless anchor.

Errors have the stable shape `{"code":"…","message":"…"}`. `DOT_NOT_FOUND`, `ANCHOR_NOT_FOUND`, and `CONTRACT_NOT_FOUND` map to HTTP 404. `INVALID_CONTRACT`, `INVALID_SELECTOR`, `SELECTOR_NOT_FOUND`, and `UNSUPPORTED_CONTRACT_FEATURE` map to HTTP 422.

Compatibility behavior

The current compatibility evaluator is an internal Registry service; there is no public compatibility endpoint yet. Its result is `COMPATIBLE`, `INCOMPATIBLE`, or `UNKNOWN`. `INCOMPATIBLE` means a deterministic mismatch was found. `UNKNOWN` means no mismatch was proved but an external or schema-aware

check is still needed.

Stable reason codes are:

- `ROLE_MISMATCH`
- `SEMANTIC_TYPES_NOT_DECLARED`
- `SEMANTIC_TYPE_MISMATCH`
- `INTERACTION_NOT_DERIVABLE`
- `INTERACTION_MISMATCH`
- `PROTOCOLS_NOT_DERIVABLE`
- `PROTOCOL_MISMATCH`
- `MEDIA_TYPES_NOT_DERIVABLE`
- `MEDIA_TYPE_MISMATCH`
- `INPUT_SCHEMA_COMPATIBILITY_UNKNOWN`
- `OUTPUT_SCHEMA_COMPATIBILITY_UNKNOWN`
- `SECURITY_REQUIREMENTS_NOT_EVALUATED`

Exact schema identifiers prove compatibility only inside the same authoritative contract UUID. Cross-contract structural schema comparison, ontology inference, transformations, and deployment-environment security satisfaction are intentionally not guessed.

Runtime handoff

`DeploymentService.anchorBindings` maps an anchor UUID to a named service port. The deployment layer then resolves the selected service and port name to a live endpoint. SFC should not copy hosts, numeric ports, URLs, credentials, certificates, network slices, or current policy state into the wire's logical anchor reference.

Reference lifecycle execution

`dotx compose apply` provides a first reference executor for the provisional composition document. It topologically orders DOT instances from explicit `dependsOn` readiness constraints while preserving document order among otherwise independent instances, supplies declared instance configuration and explicit wire materializers to the existing single-DOT deploy path, and optionally maps a named exposed anchor to a local port. Kubernetes and Helm workloads must become rollout-ready before the executor advances to a dependent instance. It does not copy those values into an Anchor or infer application configuration from a Contract.

The serialized `dot.mare6g.eu/v1alpha1` composition format has a canonical JSON Schema Draft 2020-12 document in DOT-CLI. `dotx compose schema` emits that exact schema for editor, CI, and third-party composer integration. Every composition is schema-validated before anchor, contract, deployment, and materializer resolution; unknown fields fail closed. JSON Schema defines the portable document shape, while Registry/DOT-CLI semantic validation remains authoritative for graph references and contract compatibility.

Successful applications are recorded under the dotx user cache by composition UUID. `dotx compose status` inspects each recorded runtime, and `dotx compose destroy` removes instances in reverse order and deletes state only after successful cleanup. Apply rolls back earlier instances on failure unless the operator requests `--keep-on-failure`, in which case partial state is kept. Each pending instance is recorded before runtime mutation so an interrupted apply remains discoverable for cleanup or explicit `apply --resume`. Resume requires the same composition source and runtime options, verifies the resolved plan and pinned-input fingerprint, Docker-network ownership, instance ordering, deployment selections, runtime identities, and the status of every applied instance, then retries only the trailing interrupted instance and remaining suffix. A fully applied compatible composition is an idempotent no-op; input or state drift fails closed. Dry-run mode writes no state and cannot be resumed.

Every planned DOT has a deterministic `runtimeIdentity`. For `DOCKER_IMAGE` compositions, apply uses that identity for the container name and network DNS, creates a composition-owned Docker network, and scopes named volumes per instance. Repeated instances of one Docker-image DOT are therefore isolated. Wires describe Anchor connectivity and do not implicitly impose startup order. Composition instances use explicit `dependsOn` edges for readiness constraints; document order is preserved among otherwise independent instances. This permits reciprocal and brokered connection graphs when endpoint identities are known before startup while still rejecting cycles in the explicit readiness graph. Identical environment materializations shared by several operations are coalesced, while conflicting values fail closed. Before runtime mutation, deployment also rejects any required environment declaration that has neither a DOT-declared `defaultValue` nor a resolved deploy-time override. Planning remains available for fixtures that intentionally document an incomplete integration boundary. Docker-image dry runs redact values whose DOT declarations mark them as sensitive.

Sensitive instance inputs use reference-only `secretRef` objects rather than literal values. Each reference identifies an open logical `store`, a provider-side `name`, and optional `key` and `version`; secret-store credentials remain deployer configuration. The built-in `environment` store resolves its name only at apply time. Plans and persisted composition state contain the reference, never the resolved value. This follows the explicit access boundary of the Compose Specification secrets model and Kubernetes Secret references without making either platform's resource schema part of DOT.

Docker image and Docker Compose deployments resolve the built-in `environment` store only at the final deployment handoff. Kubernetes manifest and Helm deployments accept `store: kubernetes` with an existing Secret name and key; dotx patches matching Deployments, StatefulSets, and DaemonSets through `valueFrom.secretKeyRef` without reading the value or claiming ownership of the Secret. Vagrant remains unsupported while its runtime metadata persists environment values. Unsupported stores may still be planned, but apply requires a configured resolver.

The executable slice remains intentionally fail-closed: it rejects `UNKNOWN` compatibility, readiness-dependency cycles, ambiguous deployment selection, repeated DOT instances for deployment mechanisms whose manifest-owned resource names are not yet isolated, and operation/channel values that cannot yet be derived authoritatively from the selected Contract. It also rejects a materialized wire across different runtime platforms until an explicit cross-platform network binding can establish reachability. Concrete credentials remain outside the composition state.

Known limitations and deferred work

- Compatibility is internal until its public request identity and policy inputs are stable.
- Ontology reasoning and cross-contract structural schema comparison require dedicated evaluators.
- Transformations are composer-owned and must be explicit.
- Workload identity, mTLS, OAuth scope satisfaction, remote attestation, confidentiality, network-slice, placement, latency, and availability constraints are composition/deployment policy, not anchor fields.
- Broker-channel, Unix-socket, network-device, eBPF/XDP, WASM import/export, and lifecycle-event bindings require separately reviewed polymorphic binding types. Portable object-store or artifact-registry exchange should normally remain a service Contract; local host paths remain deployment configuration rather than wires.

Acceptance material

The repository's `DOT-Testing/conformance/anchors/` directory contains the complete OpenAPI, AsyncAPI, Protobuf, GraphQL, contractless, binding, failure, and incompatibility examples. It also contains an APISIX/Keycloak scenario derived from SFC requirements while preserving the DOT/SFC ownership boundary. Run `task testing:conformance` to validate them through Java, the Registry resolvers, generated JSON Schema, and DOT CLI.

`DOT-Testing/e2e/api-gateway-oidc/` is the first opt-in live composition scenario. It models Keycloak as the provider of OIDC discovery and OAuth token operations, APISIX as the consumer of discovery and provider of a bearer-protected operation, and a headless OAuth client as the consumer of the token and protected operations. Three composer-owned wires materialize only deployment inputs; the protected wire explicitly binds the OpenAPI security scheme to RFC 6749 client credentials. `task testing:e2e:anchors:api-gateway-oidc:dry-run` exercises `compose apply --dry-run`; `task testing:e2e:anchors:api-gateway-oidc` applies the three-DOT composition, checks recorded status and native Secret references, proves anonymous access is rejected, proves token-authenticated access succeeds, and destroys the composition. The PoC-derived scenario backlog and pinned evidence revisions are documented in `DOT-Testing/e2e/USE-CASES.md`.

`DOT-Testing/composition/federation-peers/` is a read-only PoC11-derived planning and lifecycle-rendering fixture. It resolves two named instances of one federation-agent DOT plus two instances of one local-core adapter DOT. Eight compatible OpenAPI wires cover reciprocal peer receipt and the pinned implementation's Open5GS subscriber list, add, and modify operations. Each agent explicitly depends only on its local core; reciprocal peer wires do not fabricate a readiness cycle. The dry run proves scoped runtime identities, local-core-first ordering, non-sensitive local/peer node inputs, and both peer/core URL materializers without moving concrete addresses into logical anchors. Run it with `task testing:compose:federation-peers`. The source-adapted E2E under `DOT-Testing/e2e/poc11-federation-peers/` additionally builds pinned revision `303b35351697a0fd3941dccd0feff91144ef4257`, deploys two federation agents with two isolated Open5GS-compatible core doubles, and proves reciprocal far-end subscriber updates. This is implementation evidence for subscriber synchronization, not acceptance of PoC11 attestation, governed disclosure, or trust-policy enforcement.

Federated learning and artifact boundaries

The PoC2-derived fixture under `DOT-Testing/composition/poc2-federated-inference/` instantiates one Flower trainer DOT twice and connects both instances to one Flower server using the exact `flwr.proto.FlowerService.Join` bidirectional-stream selector from Flower v1.4.0's authoritative Protobuf contract. The composer materializes the provider as `host:port`, which matches Flower's client input without copying a concrete address into either anchor. Two BentoML inference anchors are exposed separately through a source-derived OpenAPI contract.

The PoC2 task also executes `compose apply --dry-run`. It proves that the two trainer instances receive unique container identities, share only their composition-owned network, resolve the Flower server through its scoped DNS identity, and do not collide on named volumes. The fixture images are normalization placeholders, so this is lifecycle rendering evidence rather than a live partner-PoC deployment claim.

The pinned PoC publishes model bundles and an atomic `current_model.json` pointer through a shared host directory. That path is declared as a service volume because it is real deployment configuration, but it is not represented as an Anchor wire. For portable cross-domain deployment, model publication should use a content-addressed artifact service such as the OCI artifact model and OCI Distribution API, optionally complemented by MLflow lifecycle metadata. The registry operation then has an authoritative service Contract and can participate in ordinary Anchor compatibility; artifact digests, provenance, and signatures remain data carried by that protocol rather than new nullable fields on `Anchor`.

Brokered interactions

For brokered protocols, the semantic producer, semantic consumer, and connection endpoint can be three different components. The PoC6-derived fixture under `DOT-Testing/composition/poc6-closed-loop/` therefore keeps distinct AsyncAPI `send` and `receive` selectors on the two anchors, selects a Mosquitto service port as the composition-owned connection endpoint, and materializes that endpoint's host and port into the actuator. The interpreter receives the same selected endpoint through a CACAO 2.0 external playbook variable at execution time, not through an invented deployment field. The MQTT address is not copied into either anchor, while the OpenC2 message shape is described once in the AsyncAPI contract. The current PoC6 fixture reconstructs that contract from pinned source evidence and does not claim it is producer-authoritative. CACAO playbooks remain orchestration artifacts rather than anchor fields. The source-adapted E2E under `DOT-Testing/e2e/poc6-cacao-openc2/` proves the chain from the pinned interpreter through MQTT/OpenC2 and the pinned actuator to an isolated NWDAF API; telemetry, AI inference, playbook selection, and a real NWDAF remain outside that acceptance boundary.

Environment-variable materializers declare whether they consume an operation URL, endpoint URL, endpoint host/port, or AsyncAPI channel address. Variable names are never treated as enough information to infer the required value. For `OPERATION_URL`, the planner resolves the effective OpenAPI server base path and selected operation path, then apply combines that contract fact with the selected deployment service endpoint. For `CHANNEL_ADDRESS`, it follows the AsyncAPI operation's local channel reference and uses the channel's `address`. The plan exposes these facts as `operationPath` and `channelAddress`; neither is duplicated in the DOT Anchor.

AsyncAPI 3 allows a Channel Object address to be absent or `null`, meaning that it is unknown.

Operations that reference the same logical channel can still be compatible in that case, but a `CHANNEL_ADDRESS` materializer is rejected because no value is available. Channel parameter defaults are applied when the contract supplies them; expressions without defaults remain unresolved.

Automatic materialization is deliberately fail-closed. Multiple distinct effective OpenAPI server paths are ambiguous. OpenAPI path templates and AsyncAPI channel templates accept explicit value-only wire `parameters` bindings such as `realm: {value: drax}`. The Contract remains authoritative for the name, location, schema, requirement, and template. DOT-CLI validates those facts, applies AsyncAPI defaults, percent-encodes values, and rejects unknown, empty, incomplete, or still-unresolved bindings. Runtime-derived values, query/header/cookie parameters, and complex serialization styles remain future adapters rather than inferred behavior.

When an application needs a derived URI rather than one of those values verbatim, a materializer may apply an RFC 6570 `uriTemplate`. The only template variable is `value`, which is the explicitly selected source value. Reserved expansion such as `\{+value}/keycloak/realms/drax` preserves an endpoint URI while adding an application-specific path. The transform belongs to the wire; neither the concrete URI nor the realm path belongs in the logical anchor.

The PoC1-derived fixture under `DOT-Testing/composition/poc1-ric-aaa/` binds the contract-declared Keycloak token operation's `realm` path parameter in a six-DOT RIC AAA feedback loop. Its separate issuer setting remains an explicit URI transform because an issuer is not the JWKS operation URL. Seven wires cover Keycloak JWKS and token acquisition, OPA decisions and deny-list updates, Config API forwarding, proxy metrics, and rogue traffic. The client secret is a declared sensitive deployment input represented in the composition only by an `environment` store reference; its value never appears in the document, plan, logs, or state. All seven wires are deterministically compatible at composition planning time. The environment-independent Registry keeps the bearer-protected proxy wire `UNKNOWN` with `SECURITY_REQUIREMENTS_NOT_EVALUATED`. The composition resolves that reason by binding the contract's `bearerAuth` scheme to the RFC 6749 client-credentials mechanism and verifying that the consumer declares and receives its token URL, client ID, and sensitive reference-only client secret. The resulting `security.status` is `CONFIGURED`; token validity, issuer trust, and runtime authorization remain acceptance and policy concerns.

The generic APISIX/Keycloak/OAuth-client E2E supplies reusable live acceptance evidence for one concrete client-credentials deployment. The source-adapted PoC1 E2E under `DOT-Testing/e2e/poc1-ric-aaa/` goes further: it builds pinned proxy and rogue-xApp modules unchanged behind thin dRAX-SDK bootstraps and executes all seven wires against digest-pinned Keycloak and OPA. A separately labelled controller adapter observes the producer Prometheus counter and JSON-patches OPA's deny list; a separately labelled Config API double records forwarding. The test proves JWT issuer validation, an allow-to-deny transition, subsequent 403 responses, unauthenticated rejection, exact far-end forwarding counts, and secret isolation from plan and state. This is `SOURCE_ADAPTED_E2E`; it does not change the meaning of `CONFIGURED` or claim acceptance of the absent producer AAA xApp, absent RIC Config API, dRAX, or a live RIC.

The PoC5-derived fixture under `DOT-Testing/composition/poc5-security-pipeline/` demonstrates a linear multi-stage pipeline built from actual FastAPI forwarding code. Log Analysis provides its events operation to Monitoring, Assessment provides its assessment operation to Log Analysis, and Mitigation provides its operation to Assessment. The three wires materialize endpoint base URLs because the consumers append their known OpenAPI paths. Business data flows from monitoring toward mitigation, while wire `from`/`to` continues to mean contract `PROVIDES`/`REQUIRES`.

Standards-described contractless composition

The PoC3-derived fixture under `DOT-Testing/composition/poc3-insider-threat-pipeline/` uses contractless Anchors for four semantic JSON handoffs across five analytics stages. Unlike an ordinary request/response dependency, each wire explicitly selects the receiving stage as `connectionEndpoint`; this covers push/ingest dataflow without changing the logical `PROVIDES → REQUIRES` ownership rule.

That plan intentionally stops at topology and compatibility. The pinned PoC3 services do not call one another, its documented pipeline orchestrator is absent, and its unused `PREV_SERVICE_URL` settings target a nonexistent route. Sequencing and payload transformation therefore remain composer-owned behavior, not Anchor fields. A standard such as Open Workflow should be evaluated before defining a DOT-specific workflow grammar.

The PoC10-derived fixture under `DOT-Testing/composition/poc10-secure-capability-exposure/` combines one reused OpenID Connect discovery contract with four contractless wires for W3C SPARQL, OPA decision/policy APIs, and Prometheus-style metrics. The pinned PoC source identifies those interaction boundaries but does not publish supported producer-owned OpenAPI documents for them. The contractless form therefore records only explicit direction, protocol, media type, and semantic identifiers instead of fabricating a duplicate contract.

`dotx compose` checks contractless directions, protocols, and media types with the same deterministic rules as the Registry. Both sides must use the same mode: a contract-backed Anchor cannot be wired to a contractless Anchor because that would compare facts with different authority. Concrete GraphDB, OPA, Keycloak, and telemetry addresses remain deployment bindings. Secrets and sample credentials from the PoC deployment are intentionally absent from the DOT and composition documents.

The official PoC10 objective additionally covers third-party event subscriptions, network-behaviour triggers, and automated restriction. Those paths remain explicit coverage gaps until the producers publish supported operations and authoritative Contracts, at which point AsyncAPI, CloudEvents, CAMARA/Open Gateway, or 3GPP CAPIF should be evaluated before defining any new DOT-specific schema.

The PoC9-derived fixture under `DOT-Testing/composition/poc9-edge-ddos-collection/` applies the same evidence rule to a heterogeneous collection path. It composes an Amarisoft WebSocket, Open5GS AMF enrichment, InfluxDB ingestion/query, and a FastAPI export using contractless Anchors plus explicit host/port materializers. Credentials remain declared sensitive deployment configuration and never appear in the wires. The official behavioural-detection and dynamic-mitigation outcome is not claimed: Kafka is disabled and has inconsistent topic names at the pinned revision, the copied 3GPP NWDAF OpenAPI files are not wired into runtime code, classification is notebook-only, and no mitigation operation is present.

The PoC8-derived fixture under `DOT-Testing/composition/poc8-ndt-observability/` demonstrates the corresponding admission rule for observability. It connects cAdvisor's Prometheus/OpenMetrics exposition endpoint to Prometheus through a composition-owned configuration file and exposes the pinned MARE FastAPI service separately. A proposed Prometheus-to-enhanced-NDT wire is intentionally absent: the consumer hard-codes `localhost`, has no bound consumer service endpoint, and provides no explicit endpoint materializer. `dotx compose validate` therefore prevents that local

implementation detail from being represented as a portable connection. Producer-reviewed DOTs should register Prometheus' version-specific OpenAPI document when its HTTP query API becomes a composed boundary; they should not copy a reduced query schema into Anchor fields. PCAP devices, local snapshot files, unwired alert receivers, and absent mitigation remain deployment or implementation gaps rather than logical service wires.

The complementary source-adapted scenario under `DOT-Testing/composition/poc8-twin-integrity/` and `DOT-Testing/e2e/poc8-twin-integrity/` exercises the official synchronization- integrity objective without redefining a container volume as an Anchor. A thin OpenAPI 3.1 adapter invokes the pinned PoC8 `generate_hash.sh` and `check_hash.sh` files unchanged. A separate synchronization-driver DOT receives the resolved checker URL through one Anchor wire and submits a matching and a forged physical/twin snapshot pair. The exposed results Anchor must return the exact producer outcomes, including `pcap file corrupted/forged`. This promotes only the SHA-256 consistency subset to `SOURCE_ADAPTED_E2E`; AI anomaly detection, live capture, configuration containment, and mitigation remain explicit gaps.

The PoC4-derived fixture under `DOT-Testing/composition/poc4-continuous-monitoring/` extends that rule across a heterogeneous OSS monitoring stack. Five contractless wires connect node-exporter, Prometheus, Filebeat, Elasticsearch, Grafana, and Kibana using official protocol/API semantics and explicit config-file or environment materializers. The pinned PoC publishes no version-matched producer contracts, so the fixture does not fabricate OpenAPI payloads. Product-owned contracts may replace those declarations once exact versions and supported operations are reviewed.

Zeek log directories, tcpdump PCAP output, host networking, packet-capture capabilities, and `/proc` //`sys` mounts are not service endpoints. OCI bind sources cover declared host paths, while `DeploymentConfig.ociRuntime` now declares OCI `CAP_*` requirements and inherited host namespace types. A composition must grant the exact pinned set explicitly; the DockerImage adapter supports capabilities plus host network/PID/IPC and rejects other namespace or composition-network topologies. Unrestricted privileged mode is intentionally absent. `DeploymentConfig.cdiDevices` separately declares logical CNCF CDI device requirements by `vendor.example/class` kind. Composition binds each one to an exact kind-matched `vendor.example/class=device`; the DockerImage adapter emits only that validated name via `--device`. CDI specification installation and allocation remain operator/orchestrator concerns. PoC4/PoC8 capture devices can reuse this model when deployable components exist, while eBPF/XDP attachment and portable PCAP artifact exchange remain separate binding/Contract work. Keeping all of this out of `Anchor` prevents a general-purpose Anchor from accumulating Linux-specific nullable fields. The official MITM detection and automated-mitigation outcome also remains unclaimed because those source components are still under development.

The PoC7-derived fixture under `DOT-Testing/composition/poc7-trust-scoring/` tests non-admission without misrepresenting source intent. Its FastAPI trust-scoring operation is exposed as a contractless provider, but the Scapy collector only prints normalized flows to stdout and names REST, Kafka, or OTLP as future work. The valid plan therefore contains both DOTs and zero wires. A negative fixture attempts to inject `TRUST_SCORING_URL`; planning rejects it because the collector declares no such input. A future producer-reviewed FastAPI OpenAPI export, or a standard event contract such as CloudEvents/AsyncAPI, can become authoritative once the transport exists. Capture-device attachments and Python scoring plugins remain separately typed deployment/artifact concerns. The collector DOT declares `CAP_NET_RAW`, `CAP_NET_ADMIN`, and a logical CDI device kind for packet capture; its composition grants the exact capabilities and binds the

concrete CDI device name `mare6g.eu/network-interface=eth0`. A lifecycle dry run proves least-privilege Docker rendering without host networking, host device paths, or `--privileged`.

The separate executable scenario under `DOT-Testing/e2e/poc7-trust-scoring/` keeps that source gap visible while testing a standards-oriented integration. It invokes the pinned collector's packet aggregation and stdout normalization, then a declared adapter publishes the resulting batch as a CloudEvents 1.0 structured HTTP event. An explicit mediator DOT validates the event and calls the pinned FastAPI scorer through a frozen OpenAPI 3.1 Contract. Two ordinary Anchor wires materialize the mediator and scorer base URLs; no transformation syntax is added to `Anchor` or hidden inside a wire. The live test verifies the far-end Subjective Logic vector and semantic equality between the registered Contract and runtime `/openapi.json`. This is `SOURCE_ADAPTED_E2E` evidence, not acceptance of production capture/CDI, plugin governance, trust-policy quality, or the official ASPO quarantine and retraining response.

`dotx compose` validates selectors for the same four standards supported by the Registry: OpenAPI, AsyncAPI, Protobuf/gRPC, and GraphQL. Protobuf and GraphQL payloads are parsed with maintained ecosystem libraries, not repository-local regular expressions.

End-to-end DOT-to-SFC execution remains a joint external acceptance step after the SFC team implements its consumer side.